AutoCorrect2 User Manual

kunkel321 | User Manual updated 3-27-2026

Welcome

My hope is that an overview of hotstrings will help explain why HotString Helper2 and the AutoCorrect2 f() Function, as well as the many word part entries, are made the way they are—so that's where we'll start. Whether good or bad; this manual reads more like a series of journal entries than a traditional software user manual. My advance apologies for the topic jumping and the tangents. That said, I recommend reading the sections in order, since they do somewhat build on each other.

Table of Contents

Welcome 1
Brief Overview of Hotstrings 3
Uses of Hotstrings 3
Making a Boilerplate Text Expansion 3
AutoCorrect 4
Philosophy of best AutoCorrect items 4
The Original Hotstring Helper 6
HotstringHelper2 6
Boilerplate Entries 7
Following Tabs 8
Show Symbols 8
Unwrap text 9
Make Replacement Box Bigger 10
The BP and AC radio buttons 10
AutoCorrect Word Entries 11
Auto Entry of New AC Items 11
Word Analysis 12
Delta String 12
Trimming 13
Add-a-Letter 13
Add Other Letters Too 14
Hotstring Suggester Tool 14
Check For, and Capture, Existing Hotstring 15
The Comparison Word List 15

The Web Frequency Metric 15
The Row of Six Buttons 17
ChatGPT word definitions 18
Validity Checks 19
Conflict Scenarios 21
Special Situations with Word-Ending Multi-Match Autocorrect Entries 22
When Misspellings Are Tolerated 23
Using Case Sensitivity to Protect Words 24
Context Sensitive Hotstrings 24
Inevitable Errors 26
Regarding Keyboard Input Buffering 26
Regarding Confounding Mis-Typings 26
AutoCorrection Logging 27
The AutoCorrectsLog File 28
ErrContextLog File 28
AC Log Analysis 28
Other AC Log Statistics 30
Hotstring Logging Circular Problem 31
Removed Hotstrings List 32
Hotstring Rarefication 32
The f() Function 32
AutoCorrect Fixes Report 33
Some Caveats 33
The CAse COrrector tool. 35
Accented words with diacritic characters. 36
Date Tool – H 36
Printer Tool 38
Drag Tools 38
Personal Hotstrings 39
The Settings Manager Tool 39
Manual Correction Logger (MCLogger) 40
Unique Strings Extractor Tool 42
Conflicting or Nullified Hotstrings Finder Tool 43
Defunctionizer Tool 43
Color Theme Integrator Tool 44
Getting Set Up and Updating 46
Folder Structure and Files 48
TL;DR 49
End 49

Brief Overview of Hotstrings

A "hot string" is a predefined sequence of keys that, when pressed in order, will trigger some action by some software. For the purposes of this discussion, we are referring to *auto-replace* hotstrings, where the triggering characters are automatically removed. A hotstring mustn't be confused with a "hotkey combination," where multiple keys are pressed simultaneously. With hotstrings, the software—in this case, AutoHotkey—watches for the string. When the series of keys is typed by a user, the software detects the input sequence, then simulates another, associated, series of key presses which are the pre-defined expansion, or "replacement" text. AutoHotkey is unique in the PC world, partly because of its ease of use, with regard to creating custom hotstrings. Indeed, the desire to allow hotstrings to be constructed in a single line of code was a big part of the impetus for the original creation of AutoHotkey (AHK). For interested readers who are new to hotstrings, a good place to start is the AHK Hotstrings Documentation. Intermediate AHK users will know that there is a Hotstring Function that programmatically creates hotstrings at runtime. The below discussion covers the first type of hotstrings, not the use of the Hotstring Function.

Uses of Hotstrings

In AHK, a hotstring might be used to activate a block of code, but a more-frequent use is to auto-replace text. "Auto-replacement" hotstrings typically have at least two components: **Trigger** and **Replacement**. An example from the AHK docs is:

```
::btw::by the way
```

The AutoHotkey application knows that this is a hotstring definition because of the two sets of double-colons. While running this code, type "*btw*" and upon pressing an "*End Character*" such as space, period, enter, etc., it will be replaced with "by the way." To add more utility, AutoHotkey allows various options that affect how the hotstring works. The options must appear between the first set of colons. Additionally, AutoHotkey allows *in-line comments* to be added at the end of individual lines of code. In-line comments always begin with a space, then semicolon, and are ignored by the AHK application. They are only there for human consumption. Here is another example:

```
:*:fwiw::for what it's worth ; the asterisk means that no end char is needed.
```

Making a Boilerplate Text Expansion

When AHK users create their own hotstrings, they usually create boilerplate "template" items. These are chunks of text that are frequently reused and so get automated with AHK. Examples include the two above (*btw* and *fwiw*). Another example is:

```
:*:sig/::Stephen Kunkel`nAutoHotkey Enthusiast`n555-1234
```

Upon typing "sig/" the instant the slash is pressed, the software would replace the trigger with:

```
Stephen Kunkel
AutoHotkey Enthusiast
555-1234
```

Please note that the trigger string is suffixed with a slash. A suffix is not the same thing as an End Char. Indeed, no end char is needed here, because of the asterisk option. It is important for the suffix "/" to be there though, otherwise you couldn't type words beginning in "sig" because it would trigger an unwanted replacement. Having a trigger string prefix would also work (e.g. /sig, .sig, or ;sig). Rather than using part of a word as the trigger, a user might also choose an *acronym* (technically an "*initialism*") made from the first letter of each word (skae).

Another relevant feature is that multi-line expansions like the one above can be made via *Continuation Section* such as:

```
::skae::
(
Stephen Kunkel
AutoHotkey Enthusiast
555-1234
)
```

This adds more lines to your script file, but it has the effect of your code being more "*what you see is what you get*," which is nice. As we will see below, Hotstring Helper makes multiline hotstrings, by default, using a Continuation Section. It is super important to remember that, if you bulk sort your hotstrings, **DO NOT sort the Continuation Section-style hotstrings**.

AutoCorrect

Another common use of AutoHotkey auto-replace hotstrings is to have a personal *Autocorrect Library*. Long-time users of Microsoft Word will be familiar with Word's AutoCorrect feature which has a library of common typos and misspellings. When you type one of them, Word instantly fixes the error. This is not the same as *Spell Check*, which locates your misspellings (underlining them with red squiggles) and suggests corrections. AutoCorrect only works *as you type* and it makes changes without prompting the user. An AHK script file with a library of autocorrect items works essentially the same way—but it is not limited to MS Office applications. It works most anywhere that you type text on your PC.

A fundamental difference between boilerplate hotstrings and autocorrect hotstrings, which becomes relevant below, is that you *intentionally* place boilerplate items in your documents, emails, etc. You don't intentionally misspell or mistype words or phrases. AutoCorrect items are preemptively added to your system as a safety net, or failsafe for unplanned typing errors.

Many AHK users will already be familiar with the most excellent 2007 AutoCorrect.ahk script by Jim Biancolo. The list, that is written in AutoHotkey v1 code, was largely derived from Wikipedia's list of common misspellings. An updated, for AHK v2, version of AutoCorrect.ahk, by the current writer (kunkel321) added about 1200 "common grammar errors," also from Wikipedia.

Philosophy of best AutoCorrect items

What makes a good AutoCorrect item? Certainly, it makes sense to have corrections for words that are *high-frequency words* in the given language; English for our purposes here. Secondly, having corrections for likely-to-occur misspellings or mis-typings makes sense. As mentioned above, the ubiquitous 2007 AutoCorrect.ahk was based primarily on Wikipedia's lists of common errors. But where does that list come from? Of course, it's the Wikipedia Typo Team! All in all, they do a fantastic job, but let's face it: With 6.7 million articles, written using 4.3 billion words, the Typo Team can't be manually proofreading all that text. My own hypothesis is that different versions of Wikipedia are programmatically compared, and when a particular word is replaced with another similar particular word, X number of times, the different versions of the word are saved and flagged as "*frequently occurring*" typo and correction pairs. If that is how they do it, it seems like a solid approach, but I think that sometimes a person writes some really long article on an obscure topic, then someone else comes along and changes the spelling of some word that occurs X times but *only occurs in said article*. That might explain some of the seemingly obscure typo-correction pairs in the list, such as: *alsation->Alsatian, bernouilli->Bernoulli, lotharingen->Lothringen, lukid->likud, mythraic->Mithraic, papanicalou->Papanicolaou, and valetta->Valletta*.

Whatever the reason for the odd entries in AutoCorrect 2007, most of the words are good useful words. It is noteworthy that the list contains 4637 whole-word corrections. Many of those words have common root words but differ in prefixes (such as *"un-" or "anti-"* or *"re-")* or suffixes (such as *"-ly" or "-ing"* or *"-ness").* When working with simple basic AHK hotstrings (those without hotstring options), it is indeed necessary to use all those whole words. And so, including the various permutations (single/plural, etc) for these items is needed. For example, consider this item:

```
::mrak::mark
```

Typing "mrak" results in "mark." But typing "unmrak" does not result in "unmark," because by default, hotstring triggers can't be preceded by alphanumeric characters. Similarly, "mraked" doesn't get corrected, because an End Char needs to be typed to trigger the replacement. There are however, hotstring options for both scenarios.

```
:?:mrak::mark ; This word-end item corrects mrak and also unmrak.
:*:mrak::mark ; This word-beginning item corrects mrak and also mraked.
```

They can be combined as well.

```
:*?:mrak::mark
```

This is a "word middle" match and fixes "birthmark," "Denmark," "earmarked," and 234 other words. Of course, not all of the 237 potential fixes are high-frequency words. I mean, how often will a person type the word "supermarketeer?" Still, 237 potential matches for a single autocorrect entry is pretty impressive, in terms of being high-utility.

Jim recognized this potential when making the original 2007 AutoCorrect.ahk script. The script contains 83 word beginning items, that use the :*: option, 15 word ending items that use :?:, and two word middles with both options :?*:.

```
:?*:compatab::compatib  ; Covers incompat* and compat*
:?*:catagor::categor  ; Covers subcatagories and catagories.
```

I had never really expanded on this concept, until being inspired by Jack Dunning's (AHK v1) book, Beginning AutoHotkey Hotstrings: A Practical Guide for Creative AutoCorrection, Text Expansion and Text Replacement. In chapter three he introduces the reader to using a site that looks up words via wildcard matches, to see how many matches there are to various word parts. For example, referring to these two items:

```
:*?:ugth::ught
:*?:igth::ight
```

Jack writes, "*According to the word lookup site referenced above, the first line of code protects against the misspelling of 133 words through the accidental swapping of the ht. The second line adds another 887 word variations to the same AutoCorrect app.*" As pointed out by Jack (and as logically concluded), the key here; the way to make the best multi-match high-utility autocorrect items, is to identify likely misspellings that correspond to a large number of words, but that also don't inadvertently misspell other words.

In 2021 I began the process of manually assessing and converting all 4637 whole-word hotstrings, from the 2007 AutoCorrect list. I already had the excellent application, WordWebPro installed. WordWeb has a utility for doing wildcard searches, so I used that tool, manually pasting in each word, then progressively deleting and backspacing letter-by-letter from the beginnings, then the ends of each word, to see which word middles were good multi-match items. An important part of the process was to also trim off the same letters from the beginning/ending of the (misspelled/typo) hotstring trigger string, to ensure that the new, shortened, trigger didn't correspond to—and therefore inadvertently misspell—any other words. This was a mind-numbingly tedious and time-consuming process. I eventually became sufficiently motivated to create the Word Analyzer Gui (WAG) Thing. As indicated in the WAG forum post,

"The WAG tool won't convert the hotstrings for you, but it will let you simultaneously winnow-down the trigger string and the replacement string and compare each to a list of English words. The winnowing process usually involves trimming the prefix or suffix from a root word (assuming that the typo is in the root). It doesn't "intelligently" remove the same letter from each string, it just removes a letter-at-a-time".

The resulting multi-match autocorrect entries—at least the good ones—are *high-utility items*. The WAG tool itself, however, is not a *high-utility tool*. Really, its only purpose for existence is to process autocorrect items. It is a "one-trick pony" in that regard. It served its purpose well, allowing me to process all the remaining whole word items from the AutoCorrect 2007 list, perhaps 10 or 20 times faster than without it. After completing the list, for a short time, the WAG Thing fell into disuse, but as I later found more whole-word autocorrect items via internet search, ChatGPT, and personal poor typing experiences (then later the MCL tool), I found that the WAG tool was still useful from time to time. Often, when typing, I'll misspell or mistype a word, then save it as an autocorrect entry. Then later, when I have time, I'll go back to, and examine, the entry to see if it is a good candidate for a multi-match word part hotstring library entry.

As discussed below, I integrated the tool as a *sub-component of Hotstring Helper*, and got rid of the "WAG" name, because…. Well, because it's rather silly to name something "Wag" unless it has something to do with happy dogs. I guess we'll call it the *Exam Pane*.

As an example of how the "word-analyzing/exam" process is occasionally useful, while writing this article—in the first paragraph—I used the word "combination." Being a person who is bad at spelling, I typed it "combonation." I can imagine myself potentially misspelling that word the same way in the future, so I decided it is probably a good candidate for an autocorrect item. The item, written for AHK (v1 or v2) would be:

```
:: combonation::combination
```

By simply adding the options, ":*?:"

```
:*?: combonation::combination
```

This becomes a multi-match item, and matches, “combination, combinational, combinations, recombination, and recombinations.” If we trim five characters off the end, we get,

```
:*?: combon::combin
```

which potentially fixes 40 different words, according to the word list I was using at the time. Note that you can’t remove any letters from the beginning,

```
:*?: ombon::ombin
```

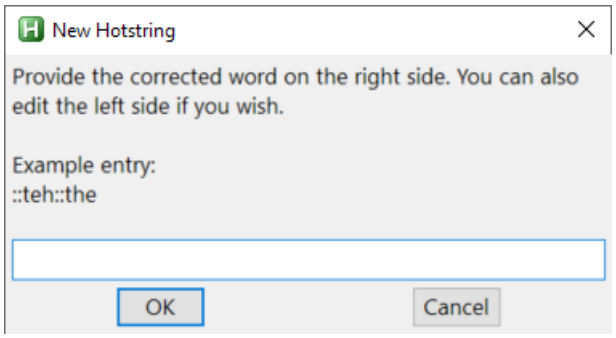
or it will inadvertently change “trombone” to the erroneous “trombine”—So that’s no good.

In addition to a simple total number of matches, (e.g. ‘fixes 40 words’) there is also the concept of which list of words has the most “high-frequency” words. That is discussed in the Web Frequency section below.

Let’s take a look at the Hotstring Helper tool, including this “combonation" example, then we’ll talk more about the f() Function and the AutoCorrect2 script, as well as a few other tools that were later added to the ac2 package.

The Original Hotstring Helper

On his blog, Jim B. credits Tara Gibb with the original Hotstring Helper tool, though the AHK Documentation suggests it was Andreas Borutta. While AutoCorrect.ahk 2007 is running, press the *Win+H* hotkey, and you’ll get what is seen in the image to the right.

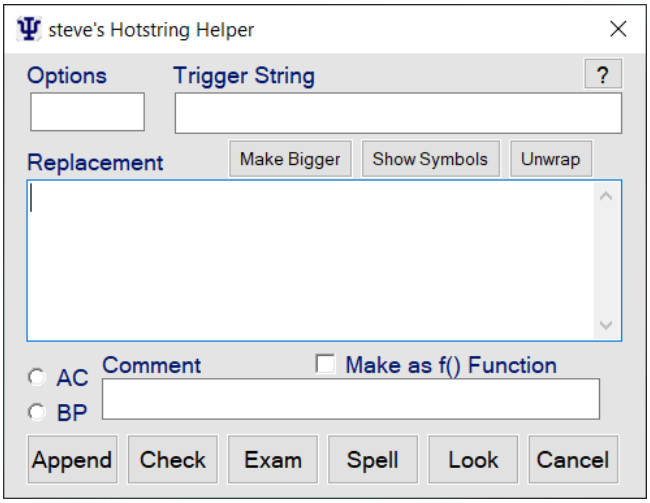


Enter your hotstring and press OK, and the new hotstring gets added to your HotstringLib.ahk file, then the file is saved and reloaded so that your new hotstring is loaded into your PC’s RAM and is ready to use. Nice! The version of Hotstring Helper from 2007 is written in AHK v1, but a v2 remake is available as well.

It’s noteworthy that AHK has a “Hotstring()” *function* which makes a hotstring immediately available to a script. Though the above-linked v2 remake utilizes the Hotstring *function*, neither the 2007 v1 version, nor kunkel321’s “ac2” uses the function. The *::trigger::replacement* style hotstrings only become available when the containing script is reloaded.

HotstringHelper2

My own version of Hotstring Helper works similarly but has several extra features. It’s noteworthy that the *“2” nomenclature* is not really a version number, per se. It’s mostly there to differentiate this version from the 2007 original. I chose “2” because it is written in AutoHotkey v2 script. “HotstringHelper2” often gets written without the “2” anyway and is often abbreviated “hh.” The original motivation for this version was to facilitate the creation of multi-line Continuation Section boilerplate items, ***in addition to*** being able to create normal single word autocorrect entries. New features have been added around this premise. The first version of this was written in AHK v1 and was called “*Hotstring Helper – Multi-Line*.” HH-ML was then remade with AHK v2 code. Upon combining the above-mentioned WAG Thing components, I considered, “*Hotstring Helper – Word-Analyzing Multi-Line*,” but that is too long, and “*WAML*” is too confusing. It appears as in the screenshot on the right. Define your own GUI form colors and icon near the top of the code. Though I don’t really use proper version numbering, there is a **version date** in the comments near the top of the code. The code has evolved over the last few years had turned into a disorganized hodgepodge of “Frankencode.” In March of 2025 I tasked Claude AI with *refactoring* the code, using best practice organization and structuring, extensibility, naming conventions, and such. I do have to say that the new version is better. Similarly, the parent folder had accumulated a couple of



dozen files, so I used Claude AI again to develop a good organization system with subfolders. That was implemented in October 2025.

The first obvious difference with between the original and HotstringHelper2 is that each component of the hotstring has its own edit box.

- Options (which are optional)
- Trigger String (a.k.a. hotstring; mandatory)
- Replacement String (a.k.a. expansion text, boilerplate text; mandatory)
- Comment (optional)

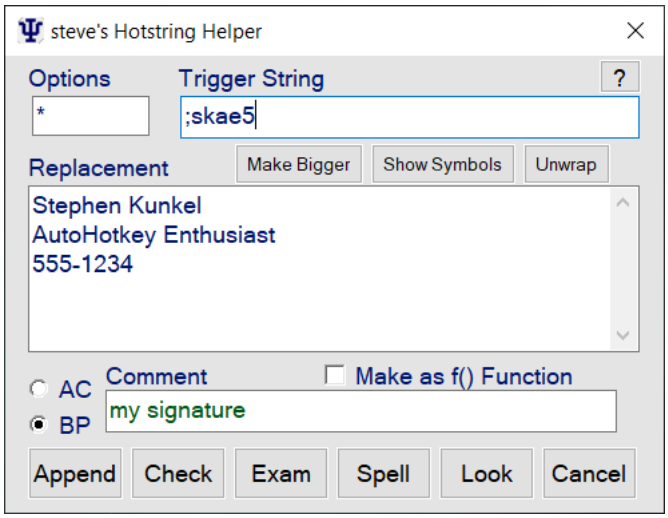
There are other components of hh2 which are discussed below. You can also select a button, box, etc. and press the F1 key for more information. Below we'll look at how hh2 is used with boilerplate entries and AutoCorrect entries.

Boilerplate Entries

My sample signature here is from the above example.

Stephen Kunkel
AutoHotkey Enthusiast
555-1234

If I select the three lines of text, then press the default hotkey: **Win+H** (technically, it's Win+h), I get what is shown in the image to the right.



Notice the automatically-generated trigger string, "**;skae5**." This is an acronym (technically an "initialism") made from the first letter of each word. It's prefixed with a semicolon because I always start my boilerplate triggers with a semicolon. The semicolon potentially confuses things, because semicolons delineate in-line comments in AHK code. They're just so convenient though… Semicolon is one of the eight Home Keys! If you have a different preferred prefix or suffix, define it with the **SettingsManager** tool – see below. When creating the acronym, hh2 uses the first character of the first X words and to skip words that are Y letters or shorter. In the [HotstringHelper] section of the SettingsManager Tool, are the relevant settings.

For the purposes of this demonstration, I manually added an asterisk to the Options edit box. This causes the hotstring to activate without the need of an End Char. Therefore, the moment I press the '5' it will trigger the replacement text. I also added a comment, again for demonstration purposes. The image to the right shows the hh2 form with these things added.

Upon clicking the Append button, the following is added to the bottom of the applicable library file:

```
:*:;skae5:: ; my signature
(
Stephen Kunkel
AutoHotkey Enthusiast
555-1234
)
```

Notes:
-Pressing the *Enter key* will also append the item. However, pressing the Enter key will not have this effect if you are editing the replacement text. When editing the replacement text, the Enter key merely does its default behavior of typing a new line.
-Note also, that holding the *Shift Key* while clicking the Append button (i.e. *Shift+Click*) will save the hotstring to the Windows

clipboard instead of appending it to the AutoCorrect file. In this situation, if the Validation Warning appears, you have to keep holding the Shift key for the item to be placed on the clipboard. Note also, the item will only be on the clipboard while the hh2 form is shown. Once you close the form, then the previous Windows clipboard contents will be restored. See below for more Append button modifiers.

Following Tabs

Sometimes I like to have Tab characters to the right of the multiline items, for example these academic test areas in the screenshot:

```
Reading⟹ ⟹ ¶
Writing⟹ ⟹ ¶
Math⟹ ⟹
```

This allows me to easily add (for example) the reading, writing, and math test scores into their own column to the right of the names after expanding the boilerplate text. When you select the multiline item, and press Win+H, *if a Tab, or space, character is detected at the end of the replacement string, then the command to keep the white space is added to the hotstring*, such as:

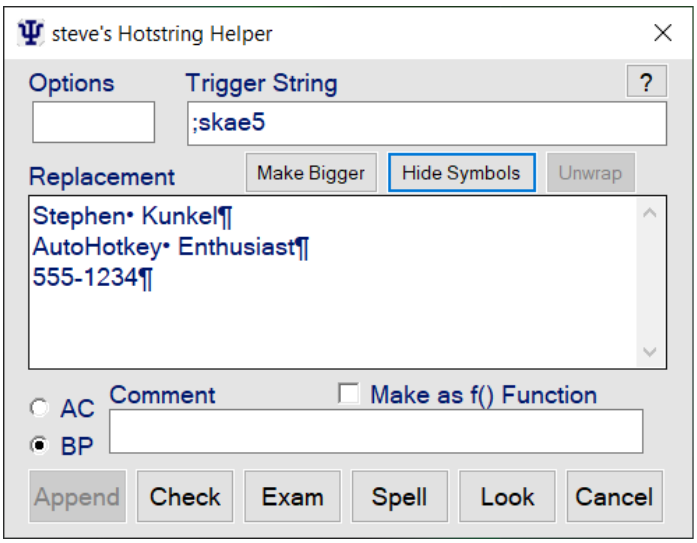
```
::;rwm::
(RTrim0
 reading
 writing
 math
)
```

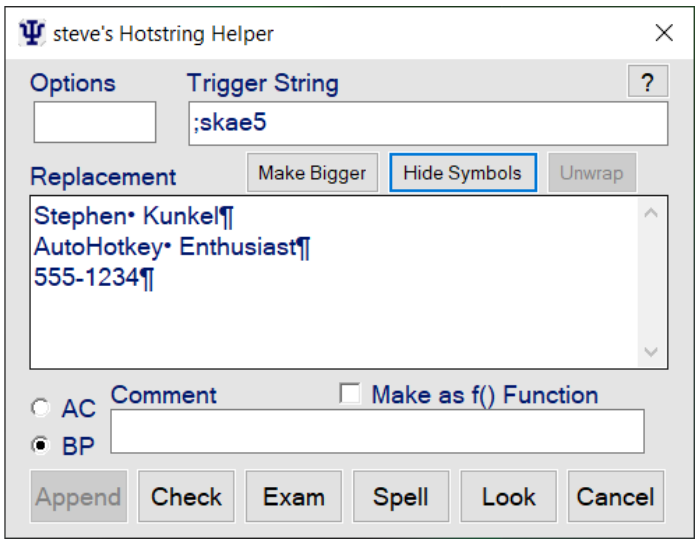
Without the *RTrim0* (Right Trim Zero, where zero means 'off'), AHK would remove the last Tab characters from each line when it expands the text. (The tabs are there; they are just invisible.)

Show Symbols

Please note the two small buttons just above the Replacement String edit box. The **Show/Hide Symbols** button toggles the visibility of a *Pilcrow, dot, and right arrow* (for Enter, Space, Tab, respectively) as in the images below. When the symbols are visible, the Replacement String box becomes *Read-Only*, and the Unwrap button and Append button become inactive. The actual symbols for these can be changed near the top of the code. The settings are:

```
; ===== Symbols for Visual Display =====
static PilcrowSymbol := "¶"     ; Symbol for Enter
static DotSymbol := "• "        ; Symbol for Space. "Dot-space" allows better wrapping.
static TabSymbol := "⟹ "        ; Symbol for Tab. "Arrow-space" allows better wrapping.
```



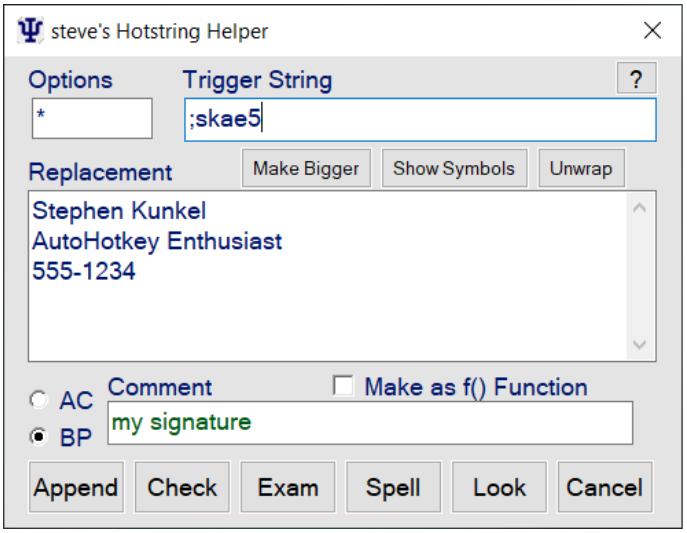


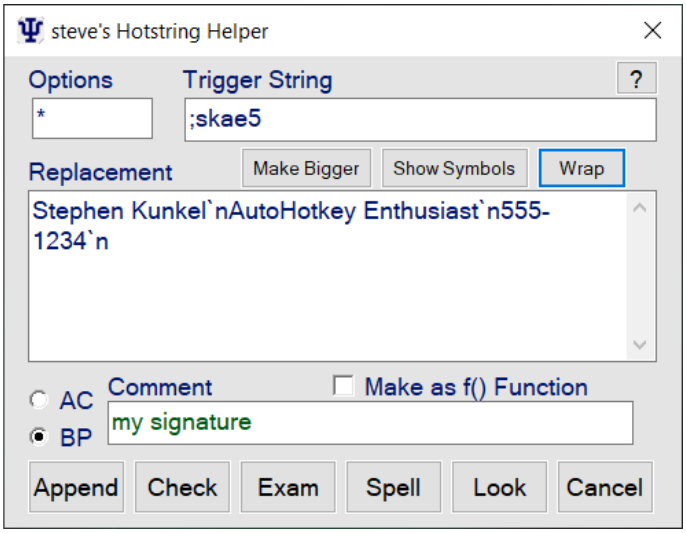
Important: Be sure to have your .ahk file formatted as *UTF-8 with BOM* or the symbols might not get displayed correctly.

Special thanks to user "Off" who helped me work out how to create this effect.

Unwrap text

As indicated in the above example, multi-line boilerplate text replacements can be encoded on multiple lines, using a *continuation section*, or as a single line of code, using *`n* characters. The **Unwrap/Wrap** button toggles this.





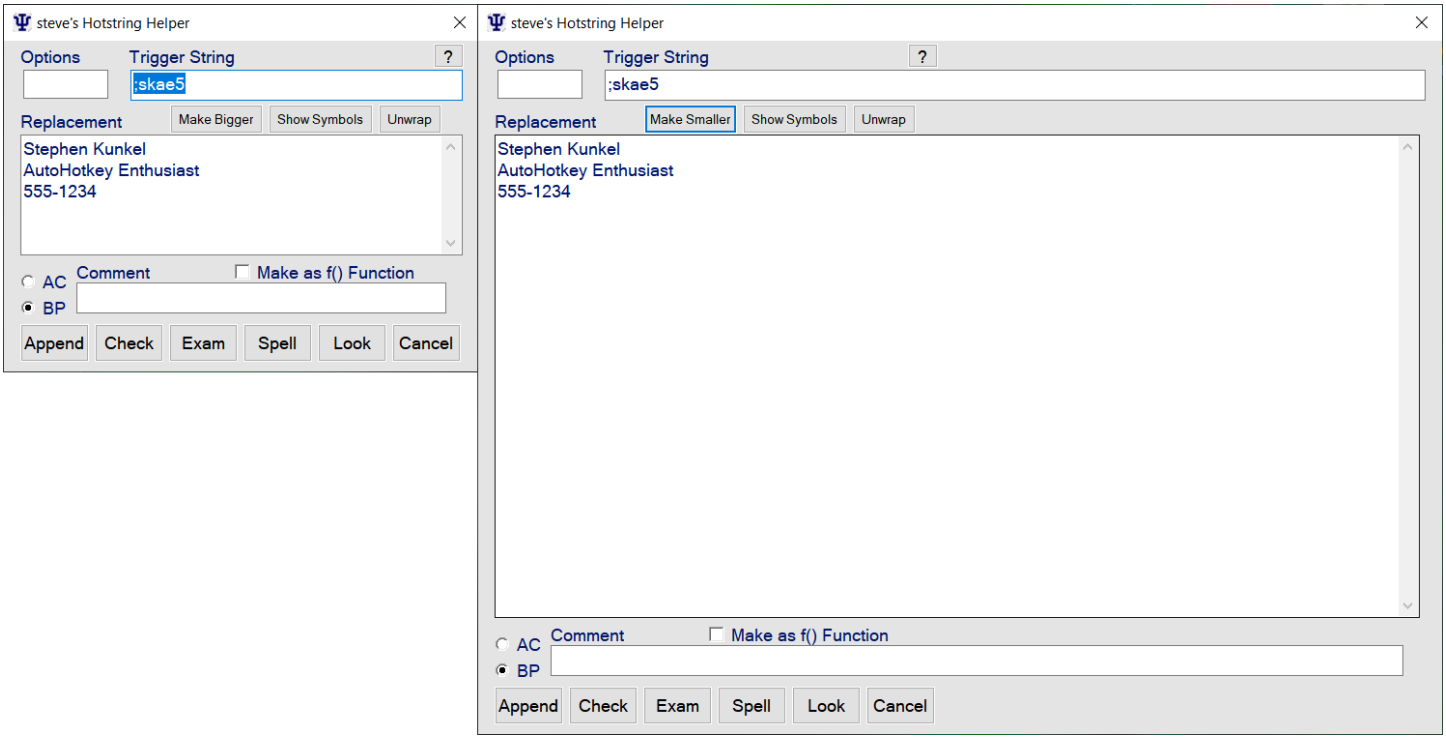
There are advantages and disadvantages to each (wrapped or unwrapped).

Make Replacement Box Bigger

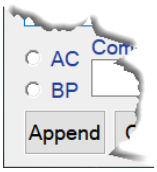
The **Make Bigger/Smaller** button toggles the size of the Replacement String box, as seen in the images below. The amount of increase can be customized with the following acSettings.ini values, in the [HotstringHelper] section. See also, Setting Manager tool.

DefaultWidth (default is 366)
HeightSizeIncrease (default is 300)
WidthSizeIncrease (default is 400)

The "DefaultWidth" value affects the width of the hh2 dialog when it is in its small state.



The BP and AC radio buttons

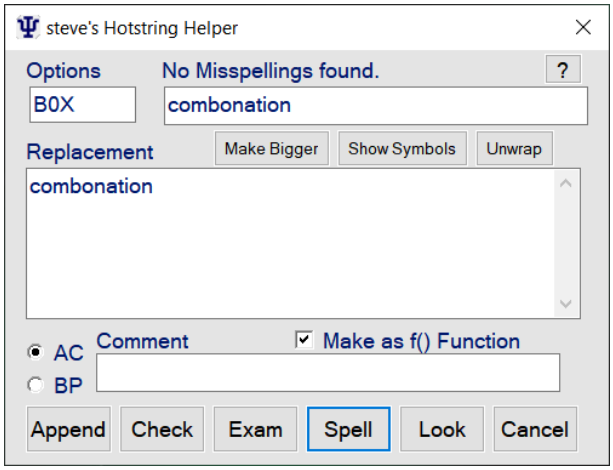


Above the *Append Button* are two radio buttons. The buttons are only present if the **SeparateLibForBoilerplates** ini key, in the **[HotstringHelper]** section is set to true/1, otherwise, they are not created. (Can be changed in Settings Manager Tool.) When selecting text and pressing the *Win+H* hotkey, if HotstringHelper detects that there are three or more words, -or- if there are multiple lines of text, then the assumption is made that the string is a new boilerplate entry, so the *BP radio button* is pre-selected. Then, when the Append Button is pressed, the new boilerplate hotstring will be appended to the **PersonalHotstrings.ahk** file, which serves as the library for boilerplate items. If SeparateLibForBoilerplates is set to false/0, then the items will be appended to the **HotstringLib.ahk** file, along with new AutoCorrect hotstring entries. If fewer than three words are selected when hh is called, then it assumes you are making a new AutoCorrect item (see next section) and the *AC radio button* is pre-selected.

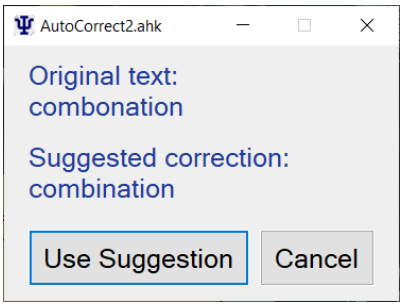
Notes:
-You can force an item to be appended to one library or the other by changing the radio button before appending. (This is the purpose of the buttons.)
-If neither radio button is selected, then the default append-to location is the HotstringLib.ahk file.

AutoCorrect Word Entries

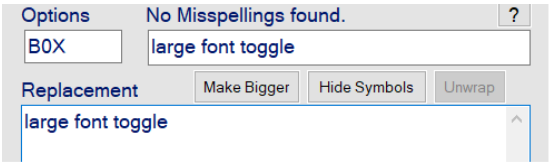
As previously indicated, the above multi-line boilerplate template features are a big part of the hh2 tool. The *other* big part is the AutoCorrect multi-match creation and entry tools. Typically, the hh2 tool is activated by first selecting a bit of text, then pressing the hotkey (Win+H by default). The tool attempts to determine if you are creating a *boilerplate item*, -or- an *autocorrect item*. There is also a *check-for-existing-hotstring* mechanism, discussed below. The check for template vs. autocorrect is as follows: If the selected text contains three or more space characters, *or* if it contains any new line characters, then assume it is a boilerplate template item and extract the initials from the first X words, of Y length. Also, if it is a boilerplate template item, use the settings defined in the ini keys, "*DefaultBoilerPlateOpts, BoilerplatePrefix*, and *BoilerplateSuffix*." If there is text selected that doesn't match this, then assume an autocorrect entry, and use the setting defined in the ini key called, "*DefaultAutoCorrectOpts*." If no text is selected, just open a blank hh2 form.



With the example screenshot on the right, I had erroneously typed "*combonation*." It occurred to me that this might be a good candidate for an autocorrect item, so I selected it with my mouse and activated hh2 via Win+H. To get the correct spelling for the replacement box, I then clicked the *Spell* button. The *Spell* tool is not a "spell checker" as commonly known. It sends the word to the free *LanguageTool.org API* and returns the suggested correction. LanguageTool doesn't always provide a suggestion, but it usually does. The suggestion appears in a message box, as seen in the image to the right. Clicking **Use Suggestion** will update the text that is in the Replacement String edit box. See below Button section for more tips.



My old eyes have trouble seeing the text in the form, so I added a Large Font toggle. Please do not confuse the function of the **Make Bigger** button, with changing font size. The font is changed in one of two ways: *(1) Ctrl+Up Arrow/Ctrl+Down Arrow* or (2) *Ctrl+Mouse Wheel Up/Down.* The default smaller size (which is seen in the top image) is s11, and the larger size is s15. This is not a gradual "zoom," it is a toggle.



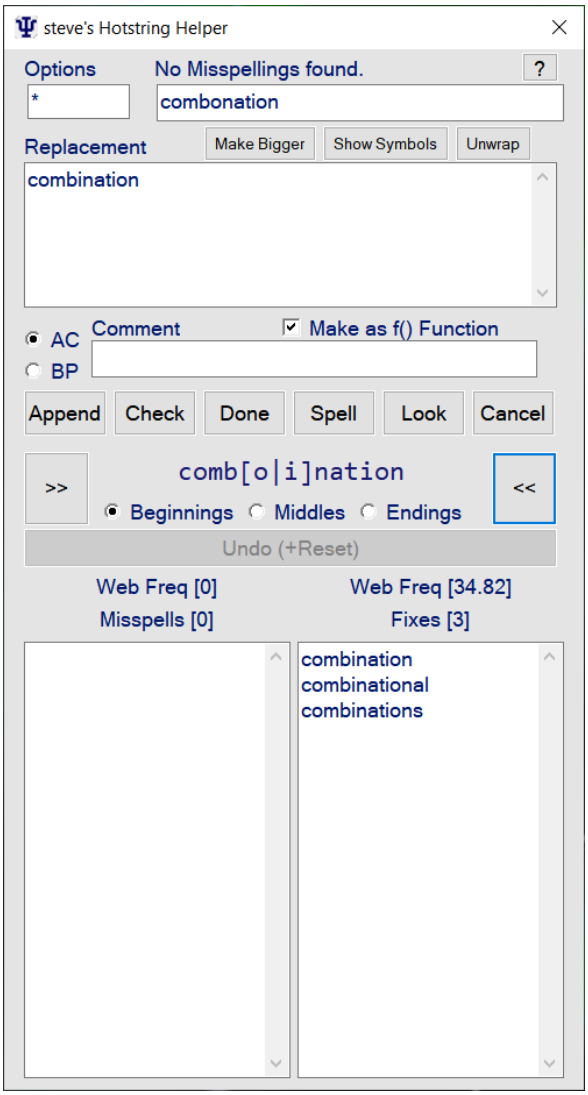
Auto Entry of New AC Items

With the above example, I had been typing a document (this document, actually) and double-clicked the misspelled word, and pressed Win+H to activate hh2. Then I corrected the spelling of the replacement string and added the item with the Append button. In a situation like this, once the hh2 form is closed, the original typo will still be selected. As such, we can automatically type the correction right into the document. The following ini key appears in the **[HotstringHelper]** section: **AutoEnterNewEntry**

If/when the value is 1/true, hh2 will remember what the active document is, then when the document is active again, it will copy the selected text a second time and compare it to the trigger of the just-created autocorrect entry. This ensures that the item in the document is still selected and the beginning/end of the hotstring item was not trimmed (i.e. to make a "multi-match" item.) If the whole trigger was used, then it is assumed that the whole *replacement* was also used, so that is typed into the document via *SendInput*, replacing the selected text. Disable this feature by setting the key to 0/false.

Word Analysis

The word "*Analyze*" is too big to fit on the button. I thought about having it say "*Anal*," (Like Anal Pane) but ultimately, I decided on "*Exam*." Click the Exam button to examine the word. When you do, the bottom part of the form, the *Exam Pane*, "pops down" and the Exam button text changes to "*Done*" as seen on the right. This incorporates the functionality of the above-mentioned Word Analysis Gui (WAG) Form.



As suggested in the above section, "*Philosophy of best multi-word matches*," the main goal here is to simultaneously *winnow-down* the trigger and the replacement words, while also testing them as word-beginnings, -endings, and -middles, to figure out what combination gives us the most "bang for our buck" in terms of being a high-utility multi-match autocorrect entry.

When the *Exam Pane* of the hh2 form is expanded, the word-beginning/ending/middle options from the Options box will be propagated down to the Exam pane radio buttons. Changing the radio buttons will, in-turn, update the Options box. Right-clicking any of the radio buttons will set them all to false/unselected, and the * and ? will both be removed from the Options box.

With my default setting of asterisk, you can see that there are already three potential words that would be corrected with the hotstring

```
:*:combonation::combination
```

The next step will be to progressively trim-off letters from the beginning and/or ending, using the **[>>]** and **[<<]** buttons. Clicking the wide Undo button (or pressing *Ctrl+Z*) will undo the trims until you are back to the original word. *Shift+Click* (*Shift+Ctrl+Z*) will undo all the trims, i.e. will reset the hotstring. You can also reset the string via *Right-Clicking* the Undo Button.

Delta String

Please note the *blue Delta string:* **comb [o | i] nation.** This serves the purpose of giving us a quick visual of how many characters can be trimmed from the beginning and end before the "typo part" of the trigger is removed. The delta string has four components:

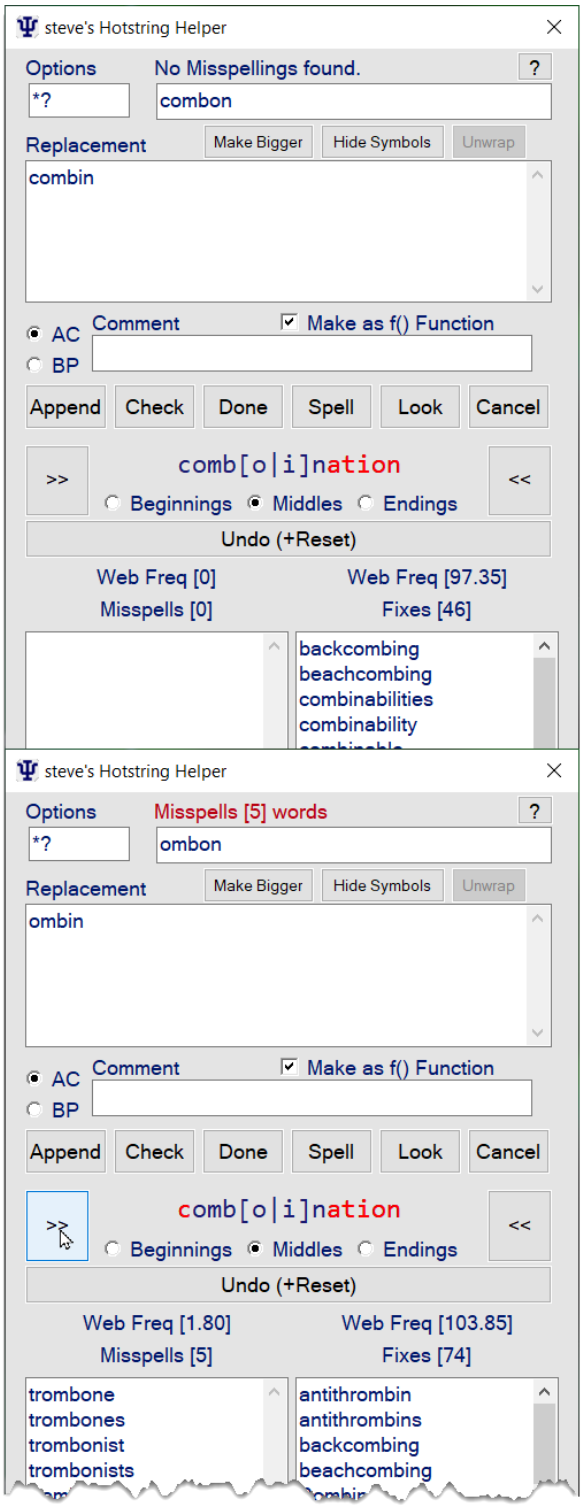
Characters that are**:**

-common to the beginning of both strings [-specific to trigger | -specific to replacement] -common to the ending of both.

Also look at this as **[the error | the fix]** and at the ends are the parts that are neither error, nor fix. The Delta string attempts to help you keep a memory of what you trimmed, via changing colors, to show the trimmed letters in red, such as: **comb [o | i] nation**. The coloring gets messed up if you trim past the bracket [ ], but in practice, a user should never do that, so it doesn't matter. Using the Undo button (or Crtl+Z) will undo the colorization too.

The bottom-most part of the form has the two lists of word matches. The words that correspond to the trigger string are on the left, and the ones for the replacement string are on the right. The goal is to have *many* words represented on the right, and *none* on the left. The example in the screenshot will potentially fix three different words and won't misspell any. If the "Misspells" box contains any words, then the Trigger String label will turn red and will show how many words are matched. This allows the user to be alerted to misspellings, even when the Exam Pane is closed. The script will assess the brightness of your form color and attempt to assign a shade of blue that contrasts with the brightness.

It is noteworthy that the Exam Pane is not useful for creating boilerplate template entries. Also, the Make Bigger feature is not useful for autocorrect entries. As such, there is no reason to have both active at the same time. Indeed, *making the hh2 form bigger will hide the Exam Pane, and vice versa.*



Trimming

With the screenshot on the right, the top part shows the effect of trimming five letters off the right side and having the *Middles* radio button selected. There are 46 potential word fixes and 0 misspellings—so far, so good.

The bottom part of the image shows the effect of then trimming one character from the left. The resulting trigger string ("ombon") is present in the word "trombone" and several other related words. So, it was trimmed too much. I clicked the Undo button, then added it to my hotstring library file with the Append button. Cool.

Add-a-Letter

Sometimes you need to add a letter back to the beginning or end. An example happened when I tried to type "Lemmings" and it got erroneously changed to "Lemings." To remedy, I selected the word, pressed Win+H, and ran a Validity Check to locate the bad entry in the library. It was this item:

```
:*?:emmin::emin
```

I like to use the word "Lemming" sometimes, so I had to fix this. Upon expanding the Exam Pane, I saw that emin fixes 299 words, but emmin is needed for "lemming," "stemming," and a few others. We can't have that. So, I visually analyzed the words that contain "emin" and found that the letter "f" precedes "emin" in about 80 of them (such as with the word "feminine"). So I got rid of ":*?:emmin::emin" and added ":*?:femmin::femin" as well as a couple of others that would correct multiple words but not misspell any. To automate the **Add-a-Letter** process a little, hh2 will watch the trigger string box, and if a letter is added to the beginning or end of the string, the same letter will be added to the replacement. An important point is that this behavior won't be desired when creating Boilerplate Template entries. As such, the Add-a-Letter feature is only available when the Exam Pane is open. The process of updating the replacement is slow, so add the letters slowly, or it won't keep up.

Important points:

- Each letter must be added to the very left or right.
- Add the letters slowly.
- Only use this feature when the Exam Pane is open.

Tip: There is a related shortcut key… Pressing **Shift+Left** focuses the Trigger string box and puts the cursor in the Home position.

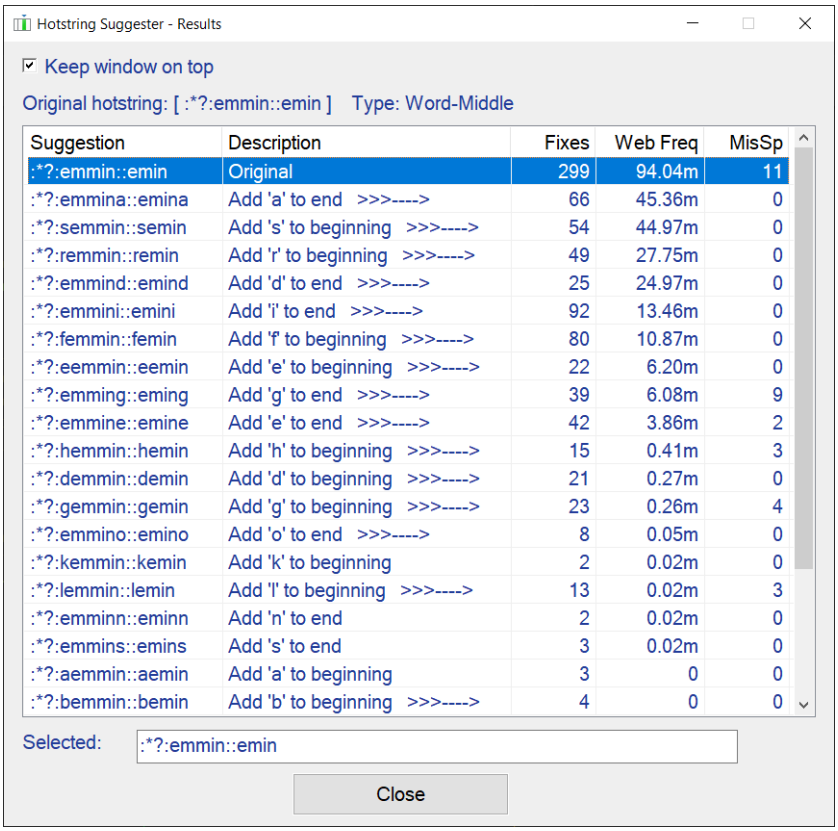
Known Problem: The add-a-letter effect is unreliable. It has worked better since the code was refactored with AI. But still, sometimes it just doesn't work. I haven't been able to figure out why. Sometimes if you close-then-open the Exam Pane, it will work again.

Add Other Letters Too

When adding-a-letter back, as discussed in the previous section, it occurred to me that I could make variations using the same "root typo" by analyzing the Fixes list and what the most common next (or preceding) letters were. This prompted the creation of the Suggester Tool. *Alt+Click* the Append button for the Suggester Tool.

Hotstring Suggester Tool

The Hotstring Suggester Tool is designed to "untrim" word middle items that got trimmed-down so far that they either have triggers that correspond to other words, or they keep resulting in confounding mis-corrections. Of course, if the item is freshly-trimmed, you can just press the Undo button. But if the item was trimmed and appended that way, there is no way to retroactively revert to the previous whole-word item. In such a case, one must view the list of Fixes to visually detect what letters correspond to words and should be added back. The Suggester Tool automates this. It can be run as a standalone tool, but it will usually be called by *Alt+Clicking* the Append button of hh2, as described above, or via an AutoCorrection Log analysis report. Essentially the tool will take the AutoCorrect item and (depending whether it is a word-beginning, -end, or -middle) will systematically try attaching each letter of the alphabet to the beginning or ending of the item. The permutations of the item that would potentially fix any words are added to the report. The number of Fixes and the number of Misspells are given. The total Web Frequency of the items in the fixes list are also given. The actual lists of words that are potentially fixed or misspelled are not given, only the metrics are shown.



Suggestion	Description	Fixes	Web Freq	MisSp
:*?:emmin::emin	Original	299	94.04m	11
:*?:emmina::emina	Add 'a' to end >>>---->	66	45.36m	0
:*?:semmin::semin	Add 's' to beginning >>>---->	54	44.97m	0
:*?:remmin::remin	Add 'r' to beginning >>>---->	49	27.75m	0
:*?:emmind::emind	Add 'd' to end >>>---->	25	24.97m	0
:*?:emmini::emini	Add 'i' to end >>>---->	92	13.46m	0
:*?:femmin::femin	Add 'f' to beginning >>>---->	80	10.87m	0
:*?:eemmin::eemin	Add 'e' to beginning >>>---->	22	6.20m	0
:*?:emming::eming	Add 'g' to end >>>---->	39	6.08m	9
:*?:emmine::emine	Add 'e' to end >>>---->	42	3.86m	2
:*?:hemmin::hemin	Add 'h' to beginning >>>---->	15	0.41m	3
:*?:demmin::demin	Add 'd' to beginning >>>---->	21	0.27m	0
:*?:gemmin::gemin	Add 'g' to beginning >>>---->	23	0.26m	4
:*?:emmino::emino	Add 'o' to end >>>---->	8	0.05m	0
:*?:kemmin::kemin	Add 'k' to beginning	2	0.02m	0
:*?:lemmin::lemin	Add 'l' to beginning >>>---->	13	0.02m	3
:*?:emminn::eminn	Add 'n' to end	2	0.02m	0
:*?:emmins::emins	Add 's' to end	3	0.02m	0
:*?:aemmin::aemin	Add 'a' to beginning	3	0	0
:*?:bemmin::bemin	Add 'b' to beginning >>>---->	4	0	0

In the screenshot, I have sorted the report by the total Web Frequency. It appears that adding an "a" to the end yields a fairly high-utility item. It would potentially fix 66 words that all share the given root. Importantly, the trigger string doesn't correspond to any words. The Web Freq value suggests that those 66 words appear on the web a total of 45.36 million times, as discussed in the Web Frequency Matrix section below.

```
:B0X*?:emmina::f("emina") ; Web Freq 45.36 | Fixes 66 words
```

Clicking a row in the Suggester output report will send the item to the Windows clipboard. Pressing *Win+H* will then open the item in hh2. Please note that the current Windows clipboard content will be overwritten.

Check For, and Capture, Existing Hotstring

The above "combonation" example involved selecting the typo with the mouse, then pressing Win+H to copy the word into the hh2 form. It takes time for you to Examine/Analyze a word though. If you are in a hurry, just use the Spell function to correct the spelling, then save the item as a whole-word autocorrect entry. *Then later…* When you have time, launch hh2 and use the *Open Hotstring Library* command in the *Secret Control Panel* (right-click Exam Button). This takes you to the bottom of your autocorrect list, where the new items are. *Select the **entire** hotstring (including colons) and press the Win+H hotkey.* There is a "*check for existing hotstring*" mechanism. If a hotstring is found in the selected text, the trigger, replacement, options, and comment will be captured, and get populated back into the hh2 form for analysis. This is done by parsing the existing hotstring with a set of *Regular Expressions*.

These particular regexes were created by Andymbody and are optimized for efficiency, utility, ease of use, and awesomeness.

The Comparison Word List

As mentioned above, I'm a big fan of the WordWeb Dictionary PC application. While creating this manual, it occurred to me that I should check with the WordWeb developer about the legality of me sharing hh2 with a word list that was generated from his (commercial) software. Indeed, I learned that the list is "non reproducible," so I've obtained several other similar, *open source*, word lists, and chose *GitHubComboList249k.txt*, which will be included in place of the list that was used with earlier versions of hh2.

The to-be-used list of comparison words gets assigned via the "WordListFile" ini key, in the Files section.

Note that the word list is also used by the ManualCorrectionLogger script. It is technically okay for the mcl and hh2 to use different word lists, but I recommend using the same list for both. This will allow the MCLogger validation feature to work better with your library of autocorrect hotstrings.

The Web Frequency Metric

As discussed in a forum post here, a new way to gauge the "utility" of a hotstring occurred to me. From the post:

> *As I was looking over the lists, it also occurred to me that certain words have a high frequency of use in written text. Words such as* the, or, and *and* are *super-high frequency words, so probably are also more important. Also, the word list that I use in hh2 to evaluate the potential number of word matches is one from GitHub that has a bit over 249k words in it. So, many of those words are incredibly obscure. (I mean, "zymogram," what even is that?) I thought surely there must be some good Open-Source lists of word frequencies online, and indeed there are.*
>
> *I decided on a list by Racheal Tatman, on Kaggle.com.*
> *https://www.kaggle.com/datasets/rtatman/english-word-frequency?resource=download*
> *The page is 8 years old, but that's okay. The Kaggle list is the "1/3 Million Most Frequent English Words on the Web." The data is apparently derived from the Google Web Trillion Word Corpus.*
> *The list is a .csv file formatted like*

```
the,23135851162
of,13151942776
and,12997637966
to,12136980858
```

> *So for example (according to Google, a few years ago), the word "the" appears on the internet 23.14 billion times. I haven't manually counted to confirm this... We'll take Google's word for it! The Kaggle list does, indeed, have 333,333 items.*
>
> *I used Claude to make a proof-of-concept script for taking a list of words and totaling the number of occurrences (the "Web Frequency") for the words. Of course, the 333k words don't line up perfectly with the 249k words in my hh2 word list.*
> *https://github.com/kunkel321/WordFrequencyTotaler*

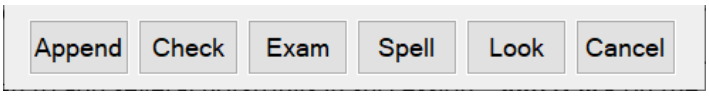
I programmatically removed any word from the 333k web frequency list that wasn't also in the 249k word list that hh2 uses. The result was the ***unigram_freq_list_filtered_88k.csv*** *file that the* WordFrequencyTotaller.ahk *script reads from.*
I also made the script accept a hotstring as an input for determining the total web frequency for the list of words that are matched by the hotstring. Only about 1/3 of the 249k words appear in the frequency data list. Presumably it is the 1/3 most common words, though, which are the ones we mostly care about.

The WordFrequencyTotaller.ahk script has been integrated into the Exam Pane of HotstringHelper2. As can be seen in the above hh2 screenshots, the web frequency (*Web Freq [#.##])* data shares a spot above each list, along with the number of *Misspellings* and *Fixes* in the edit boxes. The Web Freq numbers are expressed in millions. So the word-middle "combin" matches 46 words from the 249k word list. The total number of web occurrences for the words in that list of 46 is 97.35 million occurrences. Of course, as indicated in the quoted text, not all of the 249k words have web frequency data—only about one third of them do. No doubt all of them occur on the internet at least a few times, but Racheal skimmed off only the top 333k words from Google's Trillion. Then I removed the ones that don't appear in the 249k. Still, I think the Web Freq numbers should help give a sense of which AutoCorrect entries are likely to support more-frequently-typed words.

If we ever must choose between two conflicting hotstrings, this useful data will help us keep the hotstring with the highest utility.

The Row of Six Buttons

Of the six buttons at the bottom of the main form, several have been discussed above. Additional information is as follows:



Append: It constructs a new hotstring from the information in the form and appends it (pending validation) to the user hotstring library script, then reloads the script, so that the new hotstring is ready to use. Pressing *Enter* when the cursor is in the trigger or options boxes will also append the current item. Append Button variations:

- **Ctrl+Click** on the button will append the item, but not close the hh2 form, and not reload the script. Use this if you plan to add several hotstrings in quick succession.
- **Shift+Click** on the button will—rather than appending—construct the hotstring and save it to the Windows Clipboard. Remember that, when sending the string to the clipboard, if the Validation Message Box appears, you'll need to press Shift again, when clicking "Append Anyway."
- **Alt+Click** on the Append button will construct the hotstring and send it to the Hotstring Suggester Tool—which is a separate app that is bundled with the suite.

Check: Do a Validity Check, but don't append the item. The Check button does a *validity check* of the hotstring (whether boilerplate item, or autocorrect item), and reports information about it. Validity Checks are discussed in the next section.

Exam/Done: Opens the Exam Pane for word analysis. **Right-Click/Shift+Click** the Exam button for the ***Secret Control Panel.*** :- }

Spell: Uses LanguageTool.org's free API to attempt a spell check of the text in the replacement string box. It will also attempt to correct the grammar of a short sentence. This is useful for making grammar autocorrects such as the one below.

```
::due to it's::due to its
```

LanguageTool doesn't do well with the grammar of segments like this, however. For better results, temporarily use the segment in a short sentence such as, "This is due to it's nature." By default, the entire text of the replacement is checked, but the user can optionally selected only part of the replacement text to check.

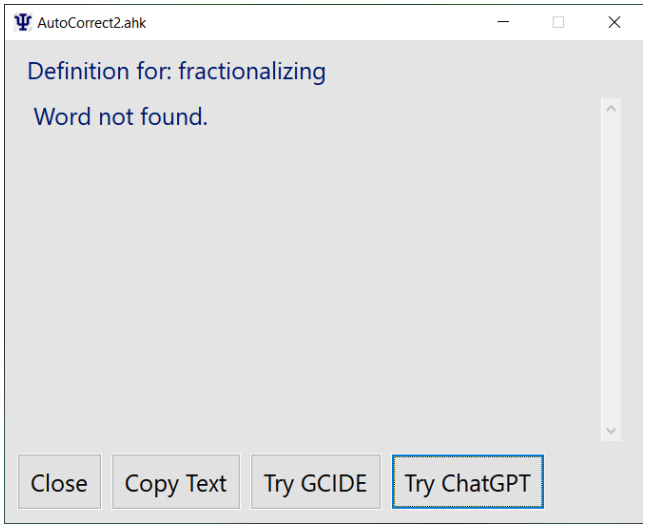
Look: This is for looking up the definitions of words in the embedded WordNet Dictionary (used legally). Select a word from one of the lists, or the replacement box and click the Look button. *The word must be selected with the mouse.* See also the ChatGPT tool below.

```
; ===== Left-Clicking the "Look" Button Uses WordNet =====
; George A. Miller (1995). WordNet: A Lexical Database for English.
; Communications of the ACM Vol. 38, No. 11: 39-41.
; Christiane Fellbaum (1998, ed.) WordNet: An Electronic Lexical Database. Cambridge, MA: MIT Press.
; Princeton University "About WordNet." WordNet. Princeton University. 2010.
; https://wordnet.princeton.edu/
; ======== Right-Clicking the Button uses GCIDE =========
; GNU Collaborative International Dictionary of English
; https://gcide.gnu.org.ua/
```

Cancel: The Cancel button simply hides the hh2 form and restores the previous clipboard contents.

ChatGPT word definitions

I found that many of the words in the 249k word list do not have definitions in the WordNet dictionary. I already had an account set up with OpenAI for another tool ('ChatGPT-AutoHotkey-Utility' by kdalanon) which requires an API key, so I modified the tool to look up words for me. If the **ChatGPTWordlookup.ahk** file is in the *Includes* folder, then a button will appear on the word definition dialog (as in the image on the right). When clicking the button, if your API key is not present, the following message will appear in the dialog box.



PersonalApiKey.ini has been created in the Data folder.

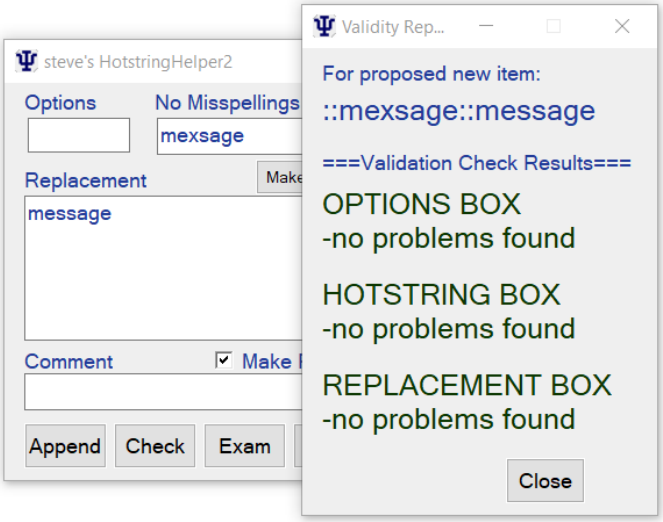
To use ChatGPT word lookup:
1. Set up an account and get an API key from:
https://platform.openai.com/account/api-keys
2. Open the AutoCorrect2\Data\PersonalApiKey.ini file.
3. Paste your API key as the ApiKey value (no quotation marks).
4. Save the INI file, then restart AutoCorrect2

Please note that the OpenAI service is not free. Also note, kunkel321 (the author of AutoCorrect2.ahk) does not receive any compensation or benefit from your transactions with OpenAI. I've only added this integration because I use it, and I thought others might like to as well. Upon testing, word lookups seem to cost a little under one cent USD each. Important: Your pre-paid tokens expire after one year! (Boooo!) As such, I don't recommend "loading up" your account with tokens. Just purchase one or two dollars worth every six months, and add a "Budget alert" for when you are about to run out.

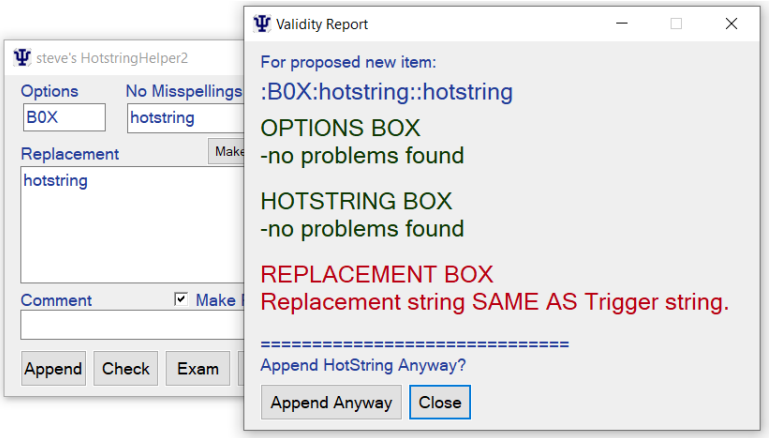
Validity Checks

The hh2 tool does several *Validity Checks*. The tool previously used an AutoHotkey MsgBox to report conflicts, but that was hard for me to see, so I made a custom dialog that was big and color-coded. Validity checks occur under two circumstances:

(1) If the user presses the Check button, a check is done and a message dialog such as the one on the right, will provide feedback as to whether there are concerns, or if the Options, Hotstring, and Replacement Boxes are all okay. With the image on the right, they are all marked with the green "no problems found." The "no problems found" text description can be customized (see in-code variable), and the size of the text can be customized (see ini key in Settings Manager tool). The dialog will attempt to adjust its own size to accommodate the validity information.

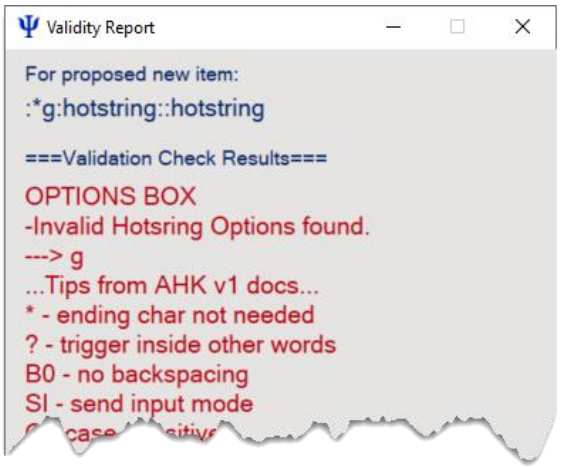


(2) If the user uses the Append button, a check is also done, but a message is *only* displayed when there are validity concerns. There is an option to "Append anyway."



The validation checks, in order of occurrence, are as follows:

OPTIONS BOX.



Valid AutoHotkey hotstring options include such things as

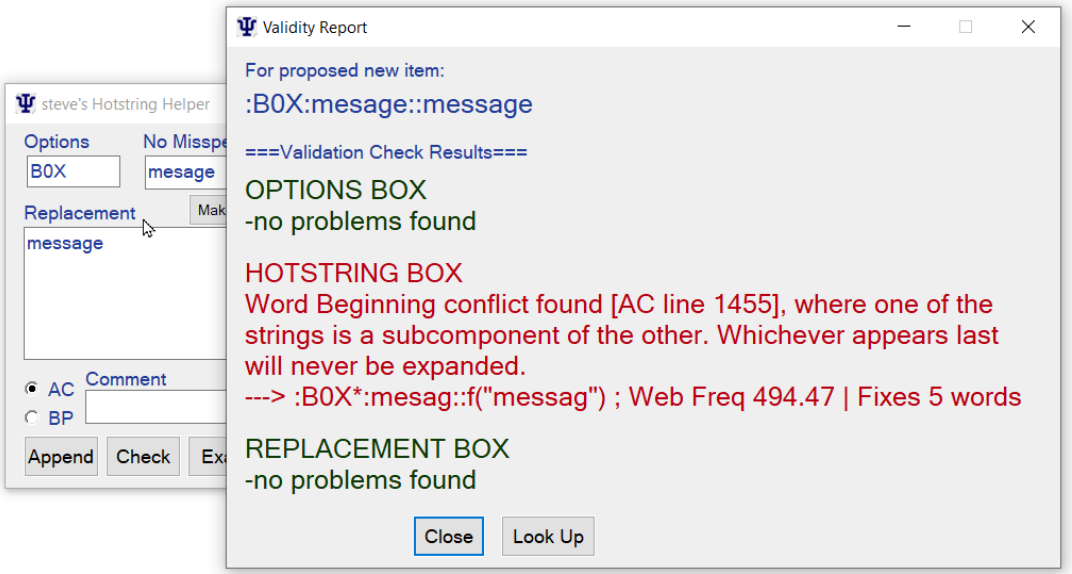
* ? B0 X T C

and appear between the first double colons. The colons themselves are not options and should not be included in the Options box. The code will flag any characters that are not valid options (such as "g" in the screenshot to the right). It will also provide some brief *Tips* that are based on the AutoHotkey Documentation. Tip: To view the hotstring options tips, just enter an invalid bit of text in the Options edit box and click the "Check" button.

HOTSTRING BOX.

Most of the validation mechanism is dedicated to checking the trigger string, i.e. the "hotstring."

*If the trigger string edit box is blank, it will warn you.
*If you are about to create an entry with a trigger that is a *Duplicate* of an existing trigger, it will warn you.
*If the new trigger string is one of the four possible subset/superset conflict scenarios (such as the screenshot on the right), it will warn you and sometimes offer suggestions. There is more discussion about these conflict scenarios below.
*If the new trigger string is likely to inadvertently misspell other words it will warn you.
*If the new trigger matches a hotstring that was previously removed, it will warn you (but only if the string is in the *Data\RemovedHotstrings.txt* list).



As seen in the screenshot, if—*and only if*—there is a problem with the hotstring which might require looking it up in your hotstring library, a button will appear to "**Look Up**." To understand the functionality of this, please note, first, that the text displayed in the OPTIONS, HOTSTRING, and REPLACEMENT BOXes can be selected with your mouse. I made it this way, because, when processing potential multi-match autocorrect entries, I found that I often want to search my AutoCorrect2 code for the already-existing, potentially-conflicting string, or go to the relevant line number. In these situations, it's much easier to copy the item to the clipboard and then paste it into the Find box of my ahk editor. In practice I find that I only ever copy text from the HOTSTRING BOX part of the validity dialog. To automate this "look up" process even further, I added the following rules: If you select a hotstring or a line number in the HOTSTRING BOX text, and press the Look Up button, then the tool attempts to open your assigned editor (as assigned via the variable in code). The hotkeys are then sent to search for the text or go to the line number. If the characters selected were digits, then *Ctrl+G* is sent, otherwise, *Ctrl+F* is sent. (Please note that these are the default hotkeys used by VSCode, other editors might use different hotkeys for "find" and "go to.") If your editor is running with Admin Permissions, but

AutoCorrect2.exe is not, then the Look Up button probably won't work. If HotstringHelper is set to use dual hotstring libraries, the correct library will be opened when the Look Up button is pressed.

REPLACEMENT BOX.

With the replacement box, the following will result in a warning:

*The string starts with a colon.
*The box is blank.
*The string is identical to the trigger string (as in a previous screenshot).

The user options for the Validity Message Dialog Box are near the top of the code, and are as follows:

```
; ===== Constants =====
static ValidOkMessage := "-no problems found"
```

A **ValidityDialogFont** ini key is present in the **[HotstringHelper]** section of *acSettings.ini*. Edit the font size via the Settings Manager Tool.

The script will assess the brightness of your form color and attempt to assign red and green colors of contrasting brightness.

Conflict Scenarios

Our previous philosophical discussion of what makes a good AutoCorrect hotstring might have also included, *"Is not a duplicate of, and does not conflict with, an existing hotstring."* Consider the following pairs of (fake/sample) autocorrect entries. Copy/paste them into a script and run it to experiment, if you want to.

```
; Word Beginning conflict (nullification?):
; wizard2 never gets used, because typing "wizer" activates wizar1.
; The order of these do not matter.
:*:wizer::wizar1
::wizerd::wizard2 ; Is nullified.

; Word Middle conflict (nullification?):
; Just like word Beginning conflict...
; experiment1 never gets used, because typing "expire" activates xperi2.
; The order of these do not matter.
::expirement::experiment1 ; Is nullified.
:*?:xpire::xperi2

; Word Ending conflict:
; Typing "likour " will result in one, or the other, WHICHEVER is first.
; The second/bottom one is never used.  Order matters here.
::likour::liquor1
:?:kour::quor2
```

First let's point out that there might be legitimate reasons for duplicate replacement strings. For example, the two next items appear in the original 2007 AutoCorrect.ahk.

```
::teh::the
::hte::the
```

These are both common ways to misspell "the." They are not duplicate hotstrings, because the triggers are different. Now consider these two:

```
::theer::there
::theer::three
```

These fix different words, so the autocorrect items, *in whole*, are not duplicates, but the duplicate *hotstrings* conflict with each other. Whichever appears first in the script file will be loaded into RAM by AutoHotkey (v1 or v2) and made available for use. The second one will be ignored. Regarding the fake items above: With the first pair, there is a word-beginning item (opts: *), whose trigger is a *substring* of the no-options one near it. Importantly, the substring is a "left-match" of the full string, i.e., the superset has additional characters *at the end* of the trigger string. These characters at the end will never be seen by AutoHotkey, because typing the first part of the trigger activates the "substring-match" item with the asterisk in the options (remember, * = no End Char is needed to activate the hotstring). The next pair has a word-middle (opts: *?), which works similarly. With either of these, the order doesn't matter. The shorter one will always supersede the longer one *unless* there is a *Context Sensitive* section in your script. We'll briefly look at Context Sensitive hotstrings below, but to conclude this section, just remember that word-beginnings and middles don't play nice with other hotstrings that are supersets (superstrings?). I'm not even sure that this should be called a "Conflict." It's really more of a "*Nullification*." Word-endings on the other hand,

```
:?:toin::tion
```

actually do conflict or compete with each other, in the sense that "*Whoever is first, wins.*" The order does matter. It is also relevant that, under certain circumstances when working with word-ending hotstrings, (1) conflicting items can both be used, and (2) potential misspellings can be nullified.

Special Situations with Word-Ending Multi-Match Autocorrect Entries

The previous section introduced how word-ending-match autocorrect items can conflict with each other in the sense that the second one is never seen. There are exceptions. Consider the below pair of items from my current AutoCorrect list.

```
:B0X?*:ngiht::f("night") ; Web Freq 376.63 | Fixes 120 words
:B0X?*:iht::f("ith") ; Web Freq 4071.87 | Fixes 774 words
```

In my own list they are embedded in f() functions (discussed below), and they are not right next to each other. They *are* in the same order though! The number of potential fixes reported will depend on the word list used. Note also that as word-endings, there would be much fewer potential fixes. The 103 and 560 totals assume that these items are word-middles. *The relevant point here*, is that ***when*** there are "overlapping" word-ending items, where one is a subset, and is a right-most-match, ***if*** the longer one is first, then they both will be active. You can paste these into an .ahk file and experiment if you want (exit AutoCorrect2 first, if it's running). Now (mis)type "goodngiht" and you'll get "goodnight." (mis)Type "wiht" and you get "with." However, if the shorter substring item is first, then the longer one doesn't work. In that case, (mis)typing "goodngiht" yields "goodngith". The hh2 Validity check mechanism is not detailed enough to recognize these intricacies. It will just report, "Word-ending conflict."

The other special situation works similarly. In this case, the longer word is one that we want to preserve and prevent it from getting misspelled. The items below are adapted from Jim B's original 2007 AutoCorrect.ahk.

```
:B0:design:: ; The B0 option turns off backspacing,
:B0:feign::  ; so the typed word just stays there.
:B0:resign::
:B0:sign::
:B0:sovereign::
{ ; <--- braces needed for v2, but not v1 AHK.
        return  ; This makes the above hotstrings do nothing
}               ; so that they override the ign->ing rule below.
:?:ign::ing
```

Notice the word-ending fix for "*ing*" at the bottom. It's interesting that there have been several articles and blog posts about the original AutoCorrect.ahk script over the years. It usually gets described as "*The script fixes some 4700 common misspellings and typos*." In reality though… If you count the ign -> ing fix, the number of corrections is much higher! Depending on what word list you compare against, the number of potential corrections might be from 7 to 15 thousand just for that one hotstring. Unfortunately, it also misspells 40 or so words, such as "sign." (Not all are shown above.) With the above chunk of code, you'll notice that, again, the longer-string-items appear first. The "B0" option tells AutoHotkey to not remove the trigger string. And since there's no replacement, the typed word just remains unchanged. Typing "ign" by itself, or following any characters that are not one

of the preceding longer strings will trigger the replacement. The longer, preceding strings are "protective" in the sense that they protect us from "miscorrecting" the words. A relevant topic that is related to this is the topic of "sacrificial words."

When Misspellings Are Tolerated

As discussed above, a good autocorrect entry corrects lots of common words, but *doesn't misspell other words* in the process. But what happens when a potential entry corrects many common words, but misspells one or two ***uncommon*** words? Do we sacrifice the uncommon word; knowing that if we ever type it, we'll have to press Ctrl+Z to undo the (mis)correction? Consider this example:

```
:*?:ahve::have ; Web Freq 1605.97 | Fixes 64 words, but misspells Ahvenanmaa, Jahvey, Wahvey, Yahve, Yahveh (All are different Hebrew names for God.)
```

The entry "::ahve::have" is one of the whole-word AutoCorrect 2007 items. By simply adding the "?*" it matches 64 English words. As it turns out, there isn't a single English word with "ahv" in it. Unfortunately, that three-letter combination does appear in several of the Hebrew names for God. If a person is a Jewish scholar (that writes in English), they will definitely be interested in this dilemma.

Here is another example:

```
:?:itr::it  ; Web Freq 7293.43 | Fixes 522 words but misspells Savitr (Important Hindu god)
```

By using the same technique that Jim did with ":?:ign::ing" we can "protect" some of these words. Unfortunately, it only works with word-endings. It does not work for word-middles or -beginnings (i.e. Ones that have an asterisk in the options.) So, this uncommon word:

```
:B0:Savitr::
```

gets protected. However the words,

```
:B0:Ahvenanmaa::
:B0:Jahvey::
:B0:Wahvey::
:B0:Yahve::
:B0:Yahveh::
```

don't get protected because the corresponding hotstring ":?*:ahve::have" doesn't allow it. There are currently (in 2026) about 42 "good" protective strings that are placed at the top of the HotstringLib.ahk file. There were about 35 that I would have included in the list, but the corresponding conflicting AutoCorrect item didn't allow them to be used. Fortunately, they really are pretty obscure words. But, as indicated in the code comments: "***If you hope to ever type any of these words, locate the corresponding autocorrect item and delete it***." As of this writing (3-23-2026), there are 142 of them (some of which are protected by the above-mentioned list). I used Claude AI to group these words by interest (for example like our above-mentioned hypothetical Jewish scholar). The lists were as follows:

- **Religion & Culture** (Jewish, Hindu, Islamic, other): *Jahvey, Wahvey, Yahve, Yahveh, Ahvenanmaa, minyanim, bayt, darshan, Dussehra, Vaisyas, swarajist, Savitr, Dormition, catechumenate, Erinys*
- **Biology & Nature** (taxonomy, botany, gardening): *Mulloidichthys, Gadiformes, Pycnanthemum, Tridacna, Actaea, Phalangiidae, Scincella, Scirpus, Illecebrum, brodiaea, Nuytsia, Dennstaedtia, Chytridiales, Synchytrium, Nummulitidae, Mephitinae, groenendael, semelparous, ampelopsis, Crataegus, heathy, grama, epigeous, hypogeous*
- **Medical & Pharmaceutical***: itraconazole, Tobramycin, hypoparathyroidism, histoincompatibility, keratomileuses, pemphigous, erysipelases, exomphalos, Premeiotic, Prepatent, Bextra, skimcoat*
- **Legal & Archaic Law:** *Repone, libelant, executrix, motu proprio, Corody, fautor, Praetor*
- **Food, Drink & Culinary Arts:** *Prosecco, semillon, gruyere, scaloppini, seviche, mamaliga, braaivleis, bialy, grama, Cachucha*
- **History, Literature & Arts:** *Akhenaton, Rabelaisian, quattrocento, yatagan, Spetsnaz, Ondaatje, Cronyn, Auchincloss, Wheatley, Gelandesprung, Airts, Taoiseach, reive, Knowe, flichter, Ressentiment, niton*
- **Science & Chemistry:** *photino, titanous, piperonal, glutaraldehydes, Minkowski, Nummulitidae, rensselaerite, Tiemannite, Accroides*
- **Foreign Language / Multilingual Writers:** *Lieve, Ciel, vicomte, pfennig, plaas, lisente, Hexerei, Gusla, motu proprio*

- **Miscellaneous Hobbyists:** *ollieing* (skateboarders), *Gelandesprung* (skiers), *braaivleis* (BBQ enthusiasts), *sinsemilla* (... certain hobbyists)

To see the definitions of these words, and the associated hotstring item that would misspell them, please use the **ExtractPotentialMisspellings.exe** app that is bundled with AutoCorrect2. There is a "Look up" button that will make it easy to find the item in the HotstringLib.ahk file. Tip: Hold the Shift key while running the app to show the list as a different format. If you want to frustrate yourself, make sure AutoCorrect2 is running, and try to type several of the above words! >:-}

It turns out that about one out of every 50 hotstrings (2%) in the library will potentially misspell some obscure word. As for the protective sub lists, both (the "good" protective strings, and the ones that aren't allowed by their respective hotstrings) were sorted alphabetically. For the above examples, I took the first item from each. It was just a coincidence that both "sacrificial" words were religious words. But… Since we're doing religious words…

Using Case Sensitivity to Protect Words

Consider this example:

```
:*C:carmel::caramel ; Web Freq 1.66 | Fixes 12 words.  Case sensitive to not misspell Carmelite (Roman Catholic friar)
```

This was made from the 2007 whole-word entry, "::carmalite::Carmelite". Believe it or not, there are 12 English words that have caramel as a root at the beginning. Adding the asterisk allows the hotstring to match any of them. Unfortunately, a Roman Catholic friar is (apparently) called a "Carmelite." I looked it up in the dictionary, and it appears to always be capitalized. So, we add the "C" option. Now if we type the word with a lower-case "c," which matches the trigger, it gets changed. If we use upper case, it does not get changed. Unfortunately, this also means that **if** I write a sentence about caramel, **and** I start the sentence with the word caramel, **and** I misspell it as the trigger string, it won't get corrected. I guess that's another "sacrifice."

For most purposes, this *case-sensitivity to protect words* technique only works with word beginnings. When the to-be-protected word is an acronym (or initialism), then word-endings or -middles might also apply. Here is an example:

```
:?C:hc::ch ; Fixes 446 words, :C: so not to break THC or LHC
```

This one is "safer" in the sense that a misspelled word at the beginning of a sentence will still get corrected; Unless, of course, we are writing in all caps. An opposite example might be:

```
:C:ASS::ADD ; Case-sensitive to fix acronym, but not word.
```

As a school psychologist who often writes reports for kids with Attention Deficit Disorder, this autocorrect can prevent a very embarrassing typo. On the other hand, if I were an inflamed internet troll, I may want to remove this item from my list, so that I could blast people, by shout-typing obscenities.

Context Sensitive Hotstrings

A couple of pages back, when discussing the problem of duplicate hotstrings, we mentioned Context-Sensitivity. AutoHotkey does allow us to create *Context Sensitive* hotstrings and hotkeys. A self-explanatory example right out of the documentation is as follows:

```
#HotIf WinActive("ahk_class Notepad")
::btw::This replacement text will appear only in Notepad.
#HotIf
::btw::This replacement text appears in windows other than Notepad.
```

The hh2 app does not create Context Sensitive hotstrings. Also, it is important to note that the above-discussed validity checking will not recognize if a hotstring is embedded between #HotIf directives. It will report, "Duplicate found," even though duplicates are okay, when one is restricted to certain contexts (which are usually a particular window). There is, however, a workaround… Just before the hotstring definitions, there is a line of code with:

```
ACitemsStartAt := A_LineNumber + 3 ; hh2 validity checks will skip lines until it gets to here.
```

Validity checks only happen below this point, so have the #HotIf items above it.

Inevitable Errors

Despite our best efforts to create hotstrings that don't have triggers corresponding to other words, there will inevitably be times when errors will occur. From my own experience, I believe there are two scenarios in which this happens the most. *Keyboard input buffering problems* and *confounding mis-typings*.

Regarding Keyboard Input Buffering

Consider this test hotstring:

```
:?*:zx::lllllllllllllllllllllllllllllllllllllllllllll
```

Put this in a script and try it. You'll notice that "zxcv" are next to each other on the keyboard, so you can type them rapidly by "strumming" the fingers of your left hand. Try it several times… This is what I get:

```
lllllllllllllllllllllllllllllllllllllclllllllllv
lllllllllllllllllllllllllllllllllllllclllllllv
lllllllllllllllllllllllllllllllllllllclllllllv
lllllllllllllllllllllllllv
llllllllllclllllllllllllllllllllllllllllv
lllllllllllllllllllllllllllllllllllllclllllllv
lllllllllllllllllllllllllllllllllllllllllllclllv
```

The AutoHotkey application is written such that hotstring execution is meant to be buffered. This means that, once the hotstring is activated, any additional keypresses should be held in RAM, then sent *after* the hotstring is finished with its own characters. The resulting expanded text *should* look like this:

```
lllllllllllllllllllllllllllllllllllllllllllllcv
lllllllllllllllllllllllllllllllllllllllllllllcv
lllllllllllllllllllllllllllllllllllllllllllllcv
lllllllllllllllllllllllllllllllllllllllllllllcv
```

Communication with the current AHK developer, *Lexikos*, suggests that other applications can compete for the keyboard buffer and cause the effect of interspersed letters. This example has a really long replacement string, but it can happen with shorter strings as well.

I find that, when expanding a boilerplate entry, I tend to sit there and wait for it to play back before I continue typing. *With autocorrects, however,* a person doesn't know when they will occur, so there isn't really an ability to pause for it… You just keep right on typing, which invites buffering glitches. As discussed in the **Log Analysis** section, below, a special tool has been created by forum member, *Descolada*, which very well addresses this input buffering dilemma.

Regarding Confounding Mis-Typings

Consider this autocorrect word-middle item:

```
:?*:cirp::crip ; Web Freq 416.51 | Fixes 146 words
```

Now imagine I intend to type "chirp," but I mistype it, "cirp." Since that error string happens to be a trigger for a different word, my AutoCorrect tool erroneously changes it to crip. *"I heard the robin crip."* I think this word-middle autocorrect entry was probably generated from the 2007 whole word item, "::scirpt::script." Note that with this scenario, the original whole-word entry :scirpt::script would not have mis-corrected the typo (it would have done nothing). However the whole-word entry will only AutoCorrect *one* word. The trimmed word-middle version will correct up to 146 words and that means more of your errors get corrected.

There's a trade-off here, shorter trigger strings (as word parts) match more words. But… longer trigger strings are less likely to be accidentally typed during a confounding misspelling. Which one should be used?

-longer strings = more unambiguous

-shorter strings = more potential matches

An argument can be made for each, but for the most part, I lean more toward shorter strings with more matches, when fine-tuning my AutoCorrect items. It's also noteworthy that shorter strings will correct the typo sooner. This is important because, if I notice the typo, I'll manually correct it. So, it's nice if the autocorrect script fixes it first—before I even notice it. This is less likely to be the case with the vanilla whole-word AutoCorrect entries. Something to consider is making items with several leading characters, but few following characters. For example,

```
:*?:attachem::attachm ; Web Freq 28.24 | Fixes 6 words
```

fixes the erroneous inclusion of an 'e' in the middle of the words. As soon as you type 'em' the 'e' disappears. Because the auto-replacement happens so soon after the typo, I might not even notice the correction (were it not for the replacement-beep). Also, having fewer following characters means that there are fewer Backspaces needed, and fewer characters need to be typed into the document, which will make the replacement more efficient. Because of the relatively large number of leading characters before the typo, it makes the trigger string unambiguous. During a recent (mid 2025) update, I went through the HotstringLib.ahk list to update the Web Freq comments in the individual hotstrings. During this update I made an effort to keep more leading characters and fewer following characters. I suppose there is also an advantage to trimming the heck out of both ends of the string, though… When there are many very short word-middle items, you can potentially get multiple autocorrections in a single long, mistyped word.

At any rate; confounding misspellings happen. With AutoHotkey's default auto-replace hotstrings as AutoCorrect items, it is difficult to keep track of which Hotstring Library items are problematic, unless you are willing to stop typing the instant a replacement happens, then try to figure out which hotstring got triggered. With thousands of hotstrings in the library, however, this could be very difficult, if not impossible. It bothers me when my AutoCorrect script makes changes and I can't keep track of, or even detect, them. I needed a way to capture the corrections.

AutoCorrection Logging

I don't know who the first person was to use *Function Calls* for individual AutoCorrect items, but I got the idea from Mikeyww. The dilemma was how to capture the most recently used trigger into a variable so that it could be viewed. The AutoHotkey application has several built-in variables (**A_Vars**), that capture various metadata about running scripts. In fact, there is an **A_ThisHotkey** variable that contains the most-recently-used hotkey. It will also work for hotstrings, but… not for *auto-replace* hotstrings, which is exactly what is needed for our purpose. So how do we collect that information? The solution is a function-calling hotstring. It won't work if you have this:

```
::tpyo::f("typo")
```

because this will only replace "tpyo" with the text *"f("typo")."* However, adding an "X" in the options tells AutoHotkey to treat the "replacement" section as computer code. It's "X for eXecute."

This version:

```
:X:tpyo::f("typo")
```

Will indeed call the function (assuming there is a function f() defined somewhere). Remember that we want to capture that last used hotstring trigger. By adding the X option, this is no longer an *autoreplace* hotstring, so we can send the variable content to the function to be used.

```
:X:tpyo::f("typo", A_ThisHotkey)
:X:hello wrold::f("hello world", A_ThisHotkey)
f(replacement, trigger)
{   Send replacement
    Last_used_trigger := trigger
}
+!F3:: MsgBox Last_used_trigger
```

Now press **Shift+Alt+F3** to see the last used hotstring trigger. Nice. It occurred to me soon that I needed at third parameter, to hold the *End Character*, so that it could be Sent, following the replacement text. Rather than simply hold (only) the last-used trigger string, it also occurred to me to append them to a Log File for later analysis. This is discussed below, and further down, we take another look at the f() function.

The AutoCorrectsLog File

When an auto replacement is made, the function (optionally) beeps, then waits for one second, and logs the item by appending it to the bottom of the *AutoCorrectsLog.txt* file. One line per item, formatted with the date as YYYY-MM-DD. The reason for waiting one second is to determine if the user presses the Backspace key. This information is recorded in the log file. If there is a normal double-hyphen "--" separating the date and string, then Backspace was not pressed during the one-second timeout. If there is a left arrow "<<" then Backspace *was* pressed.

Example:

```
2024-01-19 << :?*:voiu::viou
2024-01-19 -- :?*:cuas::caus
```

The assumption here, is that, if an autocorrect item "goes rogue" then you'll immediately press Backspace to correct it. Unfortunately, this is not a perfect system. For example, when I'm tired and busy and distracted, my typing can get pretty sloppy. It is entirely possible that the autocorrect will make the desired correction, but I'll also make an *additional* error—and backspace it—all within one second. Also, it's noteworthy that I tend to watch the computer screen as I type. This is true for most people, but not all. I suspect that most people who, for whatever reason, don't monitor their typing, probably won't want to use an autocorrect app.

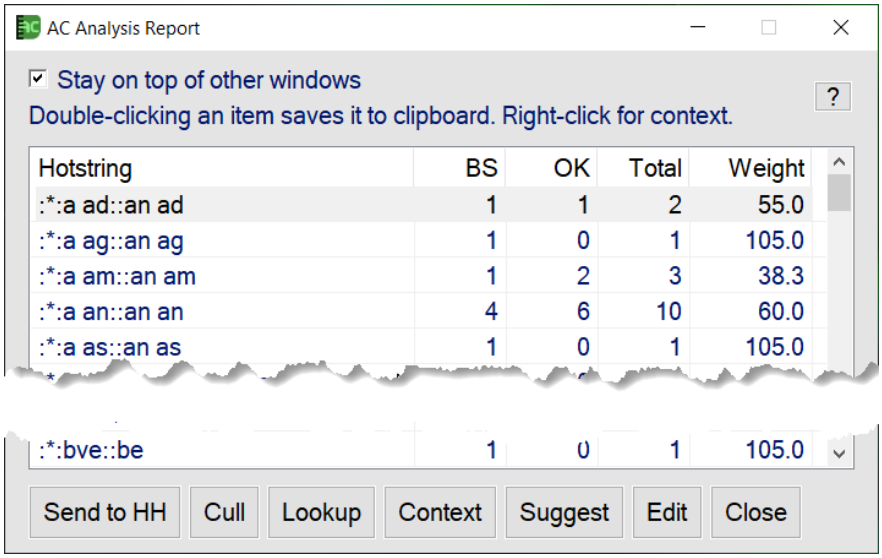
ErrContextLog File

Having the total number of times each hotstring is kept vs. backspaced is useful, but later I realized that I also want to see the context what was being typed when a particular string is corrected. Like the above *"I heard the robin crip"* Example. Is the hotstring getting triggered via repeated confounding misspellings? To help answer this question, I wanted to record a "snapshot" of the words that had been typed before and after the auto-replacement. If the context log shows "I heard the robin crip," I can infer that this was a confounded autocorrection. On the other hand, if the context log shows, "I saved the code in a script file." Then I know that the correction was applied in the correct context, so I pressed Backspace for some other reason. In the Control Pane of hh is a button, "Open Backspace Context Log." The actual file that gets opened is called *ErrContextLog.txt*. The Context Log is discussed more in the next section. It is noteworthy that the AutoCorrectsLog.txt and the ErrContextLog.txt files never need to be edited directly, or even accessed at all.

AC Log Analysis

After logging quite a few autocorrections, I wanted to generate frequency reports to analyze the data. At first I had embedded the code for that right in the log file. In Oct 2024 the log was put in a separate text file called *AutoCorrectsLog.txt.*

It is noteworthy that the *ACLogAnalyzer* script does not run in the background like AutoCorrect2 and ManualCorrectionLogger do. The analyzer must be run manually, though there is a shortcut in the hh2 Secret Control Panel (right-click Exam button). The (configurable) hotkey is *Ctrl+Alt+Shift+Q* and is defined in the acSettings.ini [Hotkeys] section.



The image to the right shows what the reported information looks like. The [?] button is for the onboard help. Also select a dialog element and press F1 for help.

By default, the items are sorted alphabetically. Click a column header in the listview to sort by that column.

The columns are:

Hotstring The item, formatted as a plain vanilla hotstring.
BS The number of times the item was followed by Backspace within one second.
OK The number of times it was logged and Backspace was not pressed within one second
Total BS + OK
Weight (BS / (BS+OK) * 100) + BS * (FreqImporance*0.1)

The **FreqImportance** variable is a value from 0-50, defined in the *acSettings.ini* file. Higher values puts more weight on items that have I large Total value. If it is set to 0, then the **Weight** is a simple ratio of Backspaced-to-total items.

If a hotstring is followed by a backspace many more times than it is not (and/or has a large weight), then it might be a good candidate for removal.

There are seven buttons on the bottom, and are as follows:

Send to HH Restarts the AutoCorrect2.exe script, sending the selected hotstring as a command line parameter. AutoCorrect2 will detect the parameter and automatically parse the hotstring and open it in HotstringHelper. **Tip:** A more efficient way to open an item in hh is to double-click the hotstring in the listview (which saves it to the Windows clipboard) then press the hh hotkey (Win+H).

Cull This will not remove the hotstring from your HotstringsLib.ahk library. It culls (removes) it from the AutoCorrectiosLog.txt file. If and item is removed from your hotstring library, then it makes sense to also cull it from the log—that way you it doesn't appear in the ACLogAnalyzer report next time.

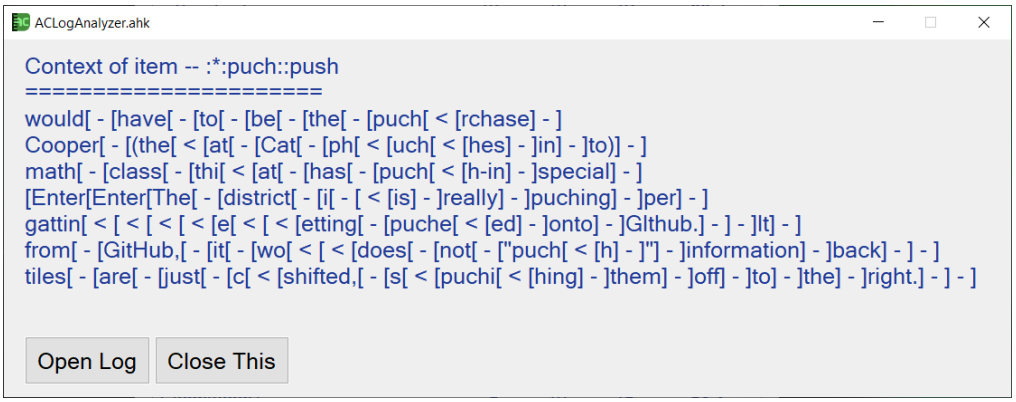
Lookup This button will attempt to open the HotstringsLib.ahk file in your AHK editor and invoke the Find tool (via Ctrl+F) and paste the string in the Find tool. If the editor takes a long time to open, then the hotstring might not get pasted in. It this happens, just click on the hotstring in the listview and press the Lookup button a second time.

Context The selected hotstring will be searched for in the ErrContextLog file and any items will be displayed in the dialog, as seen in the below screenshot. **Tip**: Right-clicking a listview item has the same effect as pressing the button.

Suggest Opens the hotstring in the Suggester Tool.

Edit Opens the *ACAnalysisReport.ahk* code for editing. Defunct since the user config options are in the *acSettings.ini* file now, rather than in the code.

Close Simply Closes the report dialog.



The screenshot to the right shows the context information for the :*:puch::push hotstring. In this dialog, the hyphens " - " indicate a space character and " < " indicates that Backspace was pressed. The words are wrapped in brackets to make them easier to distinguish. The Context Logger keeps a short cache of words in memory. The number of words kept can be defined in the Settings Manager tool, in the *ACSystem* section. The default is 8 words. When a new word is cached, the oldest is dropped, such that only the 8 latest words are ever cached. Digits and non-letter symbols are never cached. If Backspaced is pressed within one second of an auto-correction, then a "snapshot" of the words is kept. The Backspace also triggers the code to capture the next 6 (also configurable) words and record them along with the snapshot of the cache. If the required number (6) of following words are not typed, then the mechanism times-out after 12 (configurable) seconds, and the context log entry is recorded. Notice in the screenshot that the preceding (cached) words are separated with open-brackets "[" and the words recorded immediately after the backspace have close-brackets "]". Note that because of SendLevel issues, the trigger string gets logged, but the replacement that is sent does not get logged. With the first context entry, "would[- [have[- [to[- [be[- [the[- [puch[< [rchase] -]" I was attempting to type

“purchase” but missed the “r” which trigger the hotstring and changed it to “pushase,” so this was a confounding mis-correction. With the second and third items, I was typing about “push-in” special education services at school, so the corrections were the ones I wanted, but I pressed Backspace anyway. This was probably because I saw myself make the typing error and was already in the process of pressing Backspace when the correction happened. In those instances, the AutoCorrect item did not misbehave.

When deciding what to do with bad AutoCorrect entries, it is noteworthy that items can sometimes be *changed* rather than being fully removed. For example, I used to have the word-middle item :*?:daty::day. The analyzer indicated that it had been removed seven times and only kept once. Presumably, it was getting triggered when it was not supposed to. That was unfortunate, given that it potentially fixed 168 different words. Rather than just deleting the item, I changed it into two separate autocorrect entries, a word-beginning and -ending:

```
:B0X?:daty::f("day") ; Web Freq 1319.61 | Fixes 48 words
:B0X*:daty::f("day") ; Web Freq 1210.81 | Fixes 72 words
```

That was many months ago. The first one (word-beginning) works well. The second one kept getting backspaced, so I did end up getting rid of it. Before removing it from HotstringLib.ahk and culling it from the AutoCorrectionLog.txt file, I opened it in the Suggester Tool and found several superset/superstring items, such as:

```
:B0X*:datyb::f("dayb") ; Web Freq 1.36 | Fixes 8 words
:B0X*:datyd::f("dayd") ; Web Freq 0.97 | Fixes 9 words
:B0X*:datyl::f("dayl") ; Web Freq 5.06 | Fixes 9 words
```

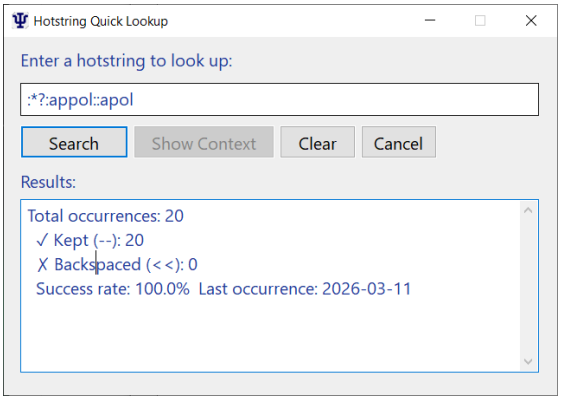
I’ve had better luck with those.

Running the analysis does take time… On my laptop, 3,760 items takes about 6 seconds.

The logging process is useful, but I have to admit that it is not the thing which has had the biggest impact on increasing the reliability of the f() system. I used to get tons of errors then, *Descolada* made his *InputBuffer Class* and shared it. This made a substantial difference, and I saw, perhaps, 80 or 90 percent fewer errors! After that, I readopted the current script and have been happy with it. The InputBuffer Class is now embedded in AutoCorrect for v2, and it gets used by the f() function. I did remove the *Mouse Buffering* elements from it, since the f() function doesn’t do anything with the mouse. The data logging and analysis are still important for finding errant hotstrings, but the buffering effect is way more important for the overall usability of AutoCorrect2.

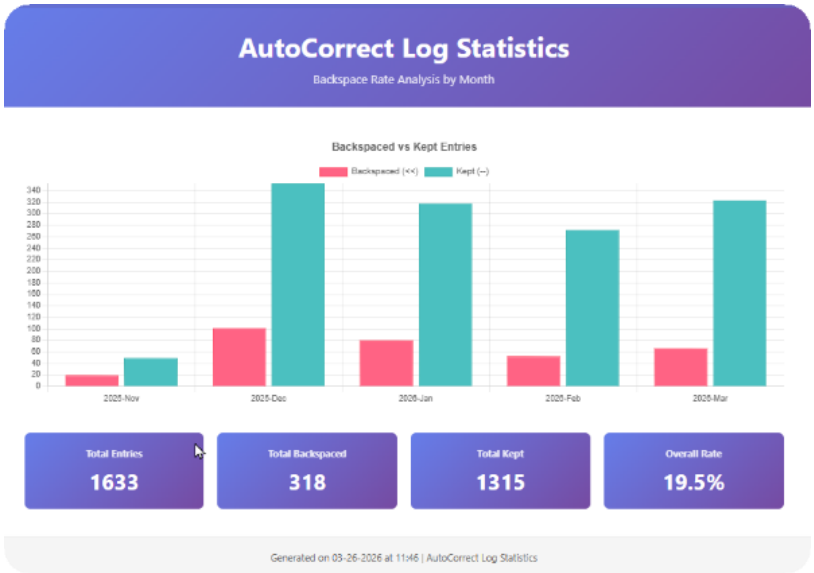
Other AC Log Statistics

Upon examining the **ACLogAnalyzer** tool, one might notice that the only items in the report are ones with at least one “<<” hyphen. What if you want to see items that have been logged and kept (“--” hyphen) but never backspaced? What if you want to see items that have never even been logged? In the *Tools* subfolder is *HotstringLibStats.exe* which runs the **Hotstring Library Statistics Tool**. This will return the log stats for all of the items in HotstringLib.ahk. By default, the list is the same as the hotstrings appear in the HotstringLib file. By and large, the vast majority of library items have never been logged. Only about 20% have log data. It is relevant though, that in May 2025, I made a major update to the library (at the same time I added the Web Freq data). At that time, I deleted my log data and started it over.



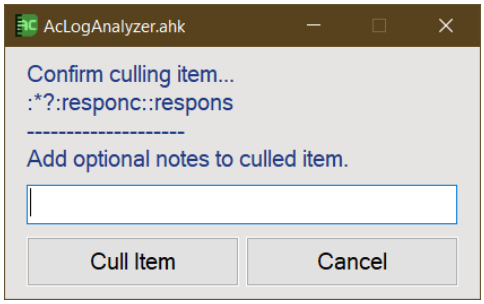
What if a person only wants log data for a single specific hotstring? For this, use the **Hotstring Quick Lookup** tool. Here is no exe file for this tool, it exists in the *HotstringQuickLookup.ahk* file, in the Includes\ *folder*, which is added to AutoCorrect2.ahk as an **#Include**. With a hotstring selected, press Ctrl+Shift+Q and get a mini-report as seen on the right. If any ErrContextLog items are found, they can be viewed too.

I also wanted a report that would allow me to view trends in my AutoCorrections over time. For example, if I am really improving my library, but removing bad entries and adding good ones, then I should see that the percentage of Kept vs. Backspaced AutoCorrections should increase over time. The **ACLogStatistics.exe** file does this. Upon running the tool, the user is asked to choose () By Month, () By Week, or () By Weekday. Incidentally, it occurred to me that each time I identify an AC item that needs to be purged from my library, I also systematically cull/remove the entry from the AutoCorrectionLog.txt file. Doing so, affectively corrupts the data that is needed by the ACLogStatistics tool. To remedy this, I needed a "static" version of the log file—One in which AutoCorrections are added, but never removed. A kind of "permanent record." To this end, I created the **ACLogContinuous.txt** file. I was hesitant to add this because there are just so many different log files in process that I was afraid it might be "overkill." Eventually I decided to go ahead and add it. The ACLogContinuous.txt file gets appended-to at the same time as the AutoCorrectsLog.txt file. The colorful image to the right shows the AC Log Statistics chart. When using the tool, an HTML document is created on demand, then the file is opened in your default web browser. The file is not actually "online" though—it is a local HTML file. Another relevant point that occurred to me later, is that I probably will never see I big improvement in the ratio of kept AutoCorrections vs backspaced ones. This is because I add new ones all the time and the new ones are untested until they are actually used in AutoCorrect2. As I remove bad hotstrings the percentages will likely get better, but then they'll decline again as a batch of new ones are added from MCLogger or other sources. For those that are curious, the bar charts shows the months Nov 2025 through Mar 2026. The ACLogContinuous.txt file was started at the end of November, which is why there are so few items. The blue/green bars show the percentage of AutoCorrections kept. The values are: 72.1%, 77.8%, 79.9%, 84.0%, 83.0%.



Hotstring Logging Circular Problem

At the time of the November 2024 update to the manual, I'd been logging and analyzing AutoCorrections for about a year and Manual Corrections for about seven months. As indicated below, the Manual Correction logger looks for words that I've manually corrected. The input hook code was logging a lot of un-useful typing data. I lessened this effect by having it check to ensure that the captured correction doesn't already exist in my AutoCorrect library. The point of logging manual corrections is to see if reoccurring typo-correction pairs should be added to my library—but there's no point adding an item that is already there. As a side note: Theoretically, if a typo has a corresponding autocorrect item, then it should never need to be manually corrected, but it does happen when the manual correction is done (by me) before enough of the characters have been typed to trigger the autocorrect hotstring. Anyway… The MCLogger tool ensures that a typo-correction pair doesn't already exist in the HotstringLib library before logging it. There was a hole in this system though. If the ACLogAnalyzer tool flagged an autocorrect item as bad, and I removed it, the MCLogger tool might then re-capture the same typo-correction pair, leading me to inadvertently reintroduce it to my library. To prevent this, I needed a way to permanently save a list of previously-removed items, so that I wouldn't keep adding (and then needing to remove) them. In Oct 2024 I added the **RemovedHotstring.txt** file. This file holds items that are culled from the AutoCorrectionLog by the ACLogAnalyzer tool. The user can optionally add an in-line note to the removed item. This can be done using the "Cull confirmation" dialog that is seen on the right. This dialog appears when the **Cull** button on the AC Log Analyzer report is pressed.



The RemovedHotstrings file is read by the MCLogger tool, to ensure that previously-removed autocorrect items are not offered-up as possible new items. It is also read by hh2's Validation Tool. If the user attempts to add a new hotstring via hh2 and the hotstring matches one in the removed hotstrings list, a warning is provided.

Removed Hotstrings List

As indicated in the AutoCorrection- and Manual Correction-Log sections, added in October of 2024, is the RemovedHotstrings.txt file. The entries are automatically generated when a hotstring is culled from the AutoCorrect log. They are formatted as:

```
Removed 10-30-2024 -> :*:alse::else
Removed 10-30-2024 -> :?:ualy::ually
Removed 10-30-2024 -> :*?:yuo::you (converted to word beginning and end items)
```

The list of previously removed items is read/accessed by the MCLogger tool and by the Validation function of HotstringHelper2. This is meant to reduce the likelihood of reoccurring problematic hotstrings being added to the HotstringLib.ahk file.

Hotstring Rarefication

While working with some of the longer "Grammar corrections" garnered from Wikipedia's list, it occurred to me that I could "**Rarify**" some of them to make them leaner… More efficient. Consider this hotstring:

```
::agreement in principal::agreement in principle
```

When auto-replacing a hotstring, the normal operation is for AutoHotkey to "Backspace away" the trigger string, then type the replacement. So, the above hotstring, as it is written, will instruct the AutoHotkey app to send Backspace 22 times, then type out the replacement. This is largely superfluous, because only the last two characters are actually different between the trigger and the replacement. So why not just press Backspace twice, then type the last two characters of the replacement string? We could turn off automatic backspacing with the "B0" (b zero) hotstring option and setup the autocorrect item like this:

```
:B0:agreement in principal::{BS 2}le
```

However, without the asterisk, an End Char is needed to trigger the string, so we need to "remember" what End Char was used and put it back. For that, we need to use the X option. Either of these will work:

```
:B0*:agreement in principal::{BS 2}le
:B0X:agreement in principal::SendInput "{BS 2}le" A_EndChar
```

The f() function, as it currently exists, will automatically, at runtime, compare the trigger and replacement strings and only remove the necessary characters. The hotstrings don't have to be set up as in the above examples. This just happens automatically every time the function is called. It's noteworthy that in about December 2023, a feature was added, by Lexikos, to AutoHotkey v2, to streamline auto-replace hotstrings in a similar way.

The f() Function

The function as it currently exists takes one parameter.

```
:B0X:trigger::f("replacement")
```

Previous versions used three parameters (as below), but upon studying Descolada's code, I saw that the second two parameters are not needed, because they are essentially like "global environmental variables" anyway, and so are available to the function without the need to send them along with the function call.

```
:B0X:trigger::f("replacement", A_ThisHotkey, A_EndChar)
```

What the function does:

- Call the InputBuffer, which captures any manual keypresses that occur during the replacement.
- Save the hotstring to a variable so we can peek at it and log it.
- Rarify the replacement and determine the number of backspaces needed.
- Type the necessary backspaces and needed part of the replacement string.
- End the Buffer, thus typing any captured manual keypresses.

- Announce the replacement with a beep (optional).
- Log the item, (via calling the GoLogger function; optional) indicating whether Backspace was manually pressed by the user within one second.

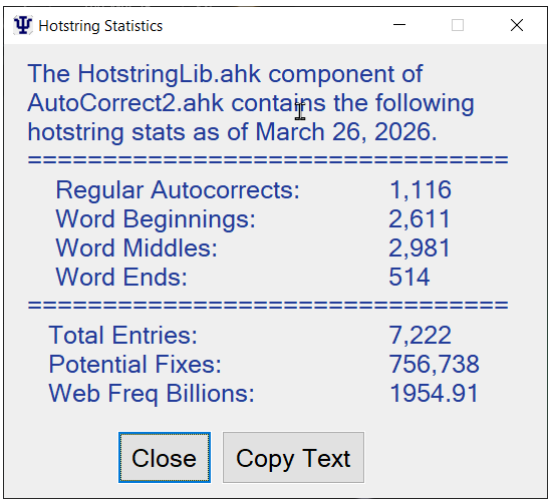
As seen in the above screenshots, there is a **[ ] Make as f() Function** checkbox under the Replacement String box, but above the Comment box. Check the box if you'd like hh2 to format the new AutoCorrect entry with the f() components. If the box is unchecked, a normal hotstring will be created. Should the checkbox be checked by default? In the acSettings.ini file, which can be edited with the Settings Manager tool, there is a key for **MakeFuncByDefault** in the [HotstringHelper] section. It is turned on by default. Turning it off will cause the *Make Function checkbox* to be *unchecked* at startup.

Related: The **EnableLogging** key in the [ACSystem] section is a master switch to disable AutoCorrection logging and Context logging.

Change that to '0' to disable all logging. As discussed above, the main reason that the f() function was even created was to facilitate the logging of autocorrections. Even if you don't want the logging capability, though, you may wish to keep the f() so as to take advantage of Descolada's InputBuffer code.

AutoCorrect Fixes Report

In the AutoCorrectSystem.ahk code is a small utility to add-up a few things. Pressing **Ctrl+F3** provides the mini-report on the right. "Regular Autocorrects" refers to whole-word items. Beginning, Middle, and End items are the titular multi-match items. When I started manually converting the 2007 AutoCorrect list, to see how many multi-word matches could be made, I flagged some of the higher-utility items with "Fixes X words" as an inline comment. Upon creating the previously-mentioned Word Analysis GUI Tool, I automated the process of adding the flag. Once practically every item was appended with its corresponding number of potential fixes, it only made sense to automate the process of tallying those. Mikeyww helped with the original scriptlet. As of this update AutoCorrect2 can potentially fix >756k items, by using 7,222 autocorrect items. The Web Freq Billions value is discussed above.



Some Caveats

* The "756k" claim doesn't mean that 756k unique words can be fixed. Indeed, the individual items were converted using the word list from GitHub. The list has about 249k words… So how can there be 756k possible fixes? It is because there are several different *and common* ways to misspell any given word. Also, one word-middle item might fix part of a word, and another might fix a different part of the same word. So, there is a great deal of overlap between the words, in terms of which words can potentially be corrected.

* Another important point is that the *actual* total number of potential matches—and—the number of possible misspellings, will vary *greatly* depending on which word list is used as a comparison. Obviously a word list with 466k words will indicate more matches than the one with 26k words. An earlier version of the HotstringLib.ahk file had "Fixes" values that were mostly from WordWeb, which has about 188k words. When updating the individual hotstring items to include the WebFreq data, I was able to reassess each hotstring using the GitHub list—So the old WordWeb data is no longer present.

* Over time, I've been frustrated by how often I find words in the 249k word list that are not real words. As mentioned in a GitHub discussion thread: Today's updated added support for ChatGPT word lookups in the Dictionary tool. I was hesitant to add the functionality, because I don't particularly want to promote a non-free service with my free product. The tool is useful, though--I think. The AI has made me realize how many non-words are in my wordlist though. :- / As mentioned in the manual, when I first made the word analysis gui tool (which got made into the Exam Pane of HotstringHelper, I set it up to include the number of potential corresponding word fixes as a comment. At the time I was using the list of words from the WordWebPro Dictionary app. When I integrated the tool into AutoCorrect2, I wanted to make sure the word list could be legally shared, so I asked the developer, and he said 'no.' So... I found a list on github, and added some words to it. There are 249,080 words in the list that is bundled with

AutoCorrect2. I integrated the open source WordNet dictionary into HH and also put a link to the GCIDE online dictionary. I find that many of the words in the 249k wordlist are not in either of those dictionaries. That doesn't surprise me though, since many of the words are obscure. What did surprise me, is that when I look them up in ChatGPT, ChatGPT also indicates that many of them are not real words. I realize the ChatGPT is not perfect, but I have to assume that it has been modeled on enough dictionaries and other things that, if ChatGPT says something is not a word, then it is REALLY not a word! Unfortunately, I'm finding that there are MANY non-words in the 249k list. I intentionally picked a big wordlist, because, when determining if a multi-match word-middle AutoCorrect item has a trigger that corresponds to any words (i.e. potential misspellings), I wanted to be very confident that there aren't any matches. Having a large list helps with this. Unfortunately, it also overestimates the number of potential fixes that are associated with the replacement strings of the multi-match items. The current stats are:

The HotstringLib.ahk component of AutoCorrect2.ahk contains the following hotstring stats as of March 26, 2026.

```
================================
  Regular Autocorrects:   1,116
  Word Beginnings:        2,611
  Word Middles:           2,981
  Word Ends:              514
================================
 Total Entries:           7,222
 Potential Fixes:         756,738
 Web Freq Billions:       1954.91
```

I suppose that many of the 756,738 potential matches are non-words. This situation poses a conundrum: How can I remove those non-words? So far, I'm just ignoring them, but while testing the new ChatGPT integration, I found so many, that I considered keeping a list, so I could go back later and remove them from the 249k list. It also occurred to me that I should try to programmatically/systematically remove them all. Presumably, I could use the same code that is in today's update, and just loop through the list, having ChatGPT assess each word. But remember that it is not free. It costs about 9/10ths of a cent (0.009 USD) per word. That might not seem like much, but for 249,080 it would cost about $2,241.00. So we won't be doing that. :- /

* It’s worth noting that I frequently reference Jim’s 2007 AutoCorrect.ahk list, which is based on the Wikipedia list, and the grammar items, based on another Wikipedia list. Indeed, perhaps 80% of the items in earlier versions of AutoCorrect2 came from those two sources. In reality though, the process of winnowing-down the words to create multi-match items loses the reference to the original word. So, my version bears little resemblance to the original lists, and it would be impossible to reverse the conversion process, to get the original items back. As you might guess, *many* of the items in AutoCorrect 2007 were also removed, because there were redundant root words with different prefixes and suffixes. It was usually the root word (or part of it) that got used for the multi-match items, so the redundant whole-word items got culled.

* It’s also noteworthy that most of the grammar-fix items are comprised of multiple words. Since the GitHub word list (which I used to calculate potential word matches/fixes) does not include multi-word phrases, there’s really no easy way to programmatically determine the number of potential fixes per item. As such, most of these items are just flagged with “Fixes 1 word.” These items also don’t have Web Frequency values included.

* Similarly, in Feb 2025, user rkingett inspired a group of hotstrings for auto-capitalizing the first letter of a sentence. For example

```
:B0X*?C:.  a::f(".  A") ; Fixes 1 word
:B0X*?C:.  b::f(".  B") ; Fixes 1 word
;...
:B0X*?C:.  z::f(".  Z") ; Fixes 1 word
```

If the user presses *period, space, space*, then a *lowercase letter*, the letter gets capitalized. Only a couple are shown here, but there are 26 letters in the alphabet, and there are three sets (period, question mark, and exclamation point). It is noteworthy that these are each flagged with "Fixes 1 word." But depending how you look at it, you could argue that each of the sets fixes any/every word. The current word list has 249k items and each of the three sets would cover every word in the list, so about 747k potential corrections.

* It is relevant that the original AutoCorrect 2007 has 262 commented-out “ambiguous” items, such as, “::wich::which, witch.” What should the replacement be? “which” or “witch?” I chose “which” because it is a word I use more often. I also picked my preferred replacement for most of the other ambiguous items as well. Many of the ambiguous entries were ambiguous because of British vs American English. My apologies go to British users who keep getting “Americanized” autocorrections.

* The list is not perfect! In the “Rarefication” section, I use the example of

```
::agreement in principal::agreement in principle
```

Upon proofreading this manual, it occurred to me that

```
::in principal::in principle
```

would be a higher-utility item because it matches “agree in principal,” and many other phrases. I searched the list, however, and found that both items, in addition to

```
::agree in principal::agree in principle
```

were already present. The “in principle” one makes the other two superfluous. There are probably other superfluous or redundant items that I haven’t found yet.

* It’s hard to search your AutoCorrect.ahk file for the typos! If I open my script, then press Ctrl+F to “find” something, then attempt to type one of the trigger strings (assuming my script is running), it will be corrected. To avoid this, type half of the trigger, then press Left Arrow, then Right Arrow, then type the rest of the trigger string. This “breaks” the hotstring recognizer sequence. Other things will do this too, such as mouse clicks or pressing Esc. You can also just copy the trigger, then paste it into the Find box with Ctrl+V. This is also the reason that the Validity dialog has a “Look up” button to automate looking up the conflicting hotstring in your library.

Some Other Items in The Suite.

Some other things present in the AutoCorrect package are as follows:

The CAse COrrector tool.

The original 2007 AutoCorrect.ahk script included a tool to *AUto-COrrect TWo COnsecutive CApitals* that was originally created by Laszo. It was a clever idea, but had problems, and so was commented-out by default in AutoCorrect 2007. As presented in the AutoCorrect for v2 thread:

I can’t be the only one who has erroneously typed “THanks" more times than I’ve typed the intended “Thanks.” It’s a useful tool! The original was made by Laszlo, using a loop to monitor for hotkeys. It was a smart idea, but would misfire under certain conditions, and so is commented-out, by default, in the original AutoCorrect.ahk script. A version posted and discussed here was made by myself (with much help) using the InputHook() function. This was much better (I’ve been using it for months) but it introduced a couple of other limitations (as seen in the forum thread). Just a couple of days ago, forum member Ntepa posted an even better version (same forum thread). His uses a hotstring loop combined with InputHook and addresses the limitations of my own version, as well as adding a couple of additional improvements. It was so superior to mine that I removed my own version and included his (with permission) in the above code. Tip: Ntepa has cleverly included a 400 ms timeout because the double-capital effect usually occurs from typing too fast. If a person wants to experiment with the tool, they might like to temporarily remove the timeout. Do this by removing the T.4 from InputHook("V I101 L1 T.4"). You could also shorten/lengthen the timeout to fit your typing/typo speed.

Unfortunately, none of the above versions are completely error free. Even with the latest version, I occasionally have a mid-sentence proper noun erroneously converted to lowercase. I had logged the active window during erroneous conversions, but there didn’t appear to be any particular applications that were correlated with the effect.

I previously had the *CAse COrrector* tool and *AutoCorrect* as separate script files for a while, but I had then re-combined them so that they could share the log function “GoLogger().” Since then, I’ve discontinued logging case corrections though, and CAse COrrector has been moved to the AutoCorrectSystem.ahk file.

Accented words with diacritic characters.

The AutoCorrect 2007 script includes 287 *accented words*, such as the example below. As with this example, many of the items have plural forms, so I made them word-beginning items. I don't consider these to be "typos or misspellings" so I didn't embed them in function-calls. I thought it would be fun to tag each item with its dictionary definition.

```
:*:angstrom::Ångström ; noun a metric unit of length equal to one ten billionth of a meter (or 0.0001 micron); used to specify wavelengths of electromagnetic radiation
```

Most of the definitions were obtained "in bulk" from https://www.easydefine.com/. Others are from the WordWeb application.

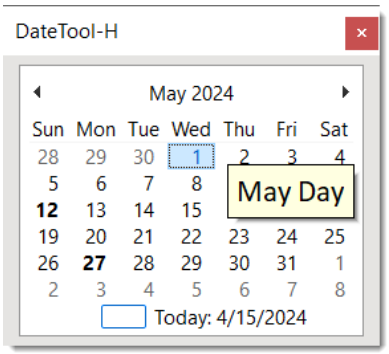
Above, we discussed the "Show Symbols" button of HotstringHelper. The *diacritic characters* in the accented words are similar to this, in that they require your .ahk file to be saved in "Unicode" format for the characters to be displayed correctly. As indicated in the AutoCorrect for v2 forum thread, I recommend this:

1) Open plain old Notepad.
2) Do "Save As"
3) Change "Save as type" from txt to "All files (.)"
4) Change the Encoding to "UTF-8 with BOM."
5) Save (using .ahk extension) /close/reopen.
6) Paste in the code that contains the accented words and save again.

The next three scripts—DateTool-H, PrinterTool, and DragTools, are not used for managing AutoCorrect hotstrings. They are extra apps that I have running every day in the background. I used to have Run commands in m ac2 script, but eventually the became "#Included." If a user doesn't care for any, or all of these three scripts, they can remove the associated #Include statements near the top of AutoCorrect2.ahk.

Date Tool – H

~ The "H" is for Holiday ;- }



The script called "*DateTool.ahk*" is a standalone tool in its own right, but it is incorporated with the AutoCorrect code byway of an **#Include** directive near the top of the code. If a person didn't want to use the date tool, they could remove that #Include line of code. There are a couple of users that like that DateTool, but don't user the other AutoCorrect2 functionality. To accommodate this, latest versions of the DateTool.ahk script are self-contained, in the sense that the .ahk script doesn't rely on acSettings.ini or have any other dependencies.

The default hotkey, **Shift+Alt+D**, shows the form as seen on the right. Yearly holidays are programmed into the code and appear as bold dates in the month calendar. Clicking on a holiday shows the name (again, see image). Pressing a number key (1-5) shows that many months. The default number of months can be set with the variable in this line of code:

```
MonthCount := 3    ; default number of months to display vertically
```

Pressing the "**h**" key toggles a list of all the holidays currently shown. They appear in a popup list as seen on the right. Pressing "**t**" takes the calendar back to "today."

Jul-04	Independence Day
Aug-25	Anniversary
Sep-02	Labor Day
Sep-22	Autumn Equinox
Oct-14	Indigenous Peoples Day
Oct-31	Halloween
Nov-11	Veterans Day
Nov-28	Thanksgiving Day

Credit for the creation of this tool goes almost entirely to AHK Forum user ***Just Me***. I saw an old post by *PhiLho*, about bolding certain days in the AHK MonthCal GUI; and another post by *Tidbit*, that featured a function to predefine yearly US Holidays, and return whether a given date was on one of them. I had added a "help" post on the v1 forums to see if anyone could combine them. Just Me did. Then later, a couple of us asked if anyone could convert the tool to AHK v2 code. Again, *Just Me stepped in* and converted the *isHoliday()* function and the MonthCal code for us. Check the isHoliday() function to see how to set up holidays. You might want to remove my wedding anniversary – LOL. I tweaked the code to allow single-occurrence events, though the tweak is

kludgy. Sometime later, I saw the ToolTipOptions Class – Again, created by *Just Me* – and added that to the bottom of the code. ToolTipOptions is what allows the nice big popup tool tips seen in the images. The font, color, and other things are customizable. Cool. "Holidays" that are one-time (non-repeating) events can also be added. See the comments in the code for instructions.

In addition to the pop-up GUI Calendar, there is a hotstring component to DateTool-H. The rule for entering dates is: ;dn. (Semi-colon, letter D, and n, a digit from 0-9, to enter a date which is n-days in the future.

Therefore:
;d0 enters the current date.
;d1 enters tomorrow's date.
;d2 enters the day after tomorrow.
etc.

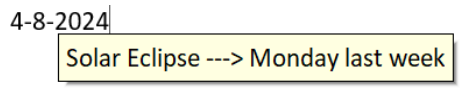
The rule for entering dates in the past, is to use negative values:
;d-1 enters yesterday's date.
;d-2 enters the day before yesterday.

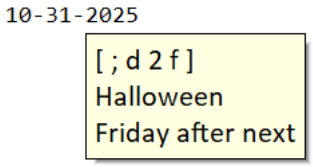
Alternatively, as a shortcut, type "double-Ds." Such as:
;dd1 enters yesterday's date.
;dd2 enters the day before yesterday
etc,

In March of 2025 forum user *CaptInsano* contributed some significant improvements to the code. This included significant expansion of the hotstring capabilities as well as a popup help dialog that is accessed via typing ;dh (that's 'h' for 'help').

A similar dialog is available for the calendar, if the F1 key is pressed when the calendar is selected. Please see these help dialogs for useful information.

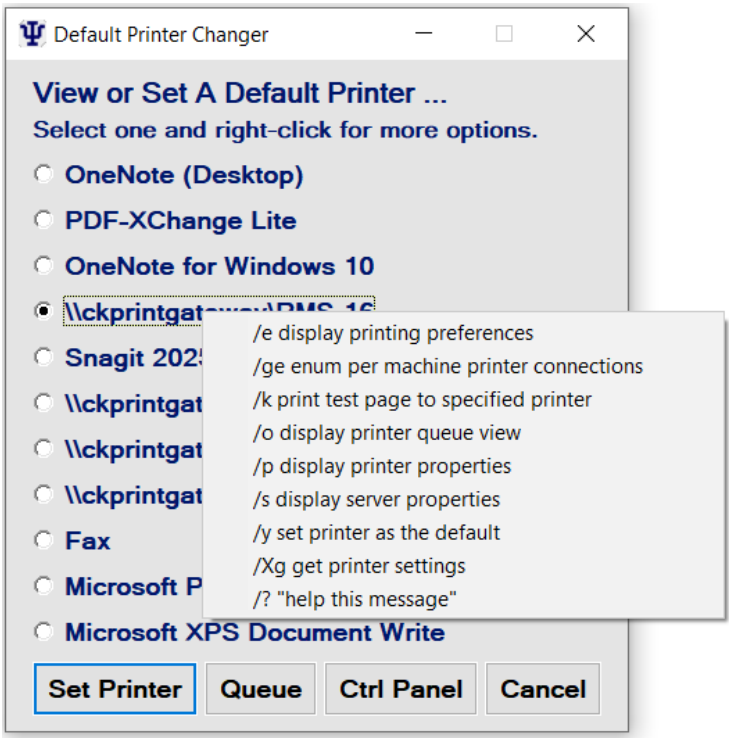
A popup tooltip appears near the entered date, telling you information about the date you've just entered. If the day was a holiday, the tooltip shows the holiday (see image). If the date falls on a weekend, that too is shown (not in image). The date formats for the date that gets entered, and for the dates in the list of holidays, can be individually defined. There are several options available regarding what should be displayed in the ToolTip.





Printer Tool

This is a simple tool. I made it so that I could—with the press of a hotkey—see what my default printer is. Thanks go to forum users Garry, Boiler, and RussF for helping. As indicated in the screenshot, the script lists your currently installed printers, making each into a radio button. The printer that is currently set as default, is indicated by the radio button being set to "True." Change default printers by selecting a different radio button item, then pressing the "Set Printer" button.



Also seen in the image, right-click on the form for a list of extra printer-related command-line functions. Please note that the options in the list will be associated with whatever printer is selected—whether or not that is the one you right-clicked on. So select the radio button for the item you are interested in, *then* right-click. Note also that not every item in the popup list will apply to every printer. Also, please note the bottom item in the list, "**help this message**." See the associated pop up for additional items you may like to add to the list. To add options, look for the PrinterToolMenu() function in the code at

AutoCorrect2\Tools\Integrated\PrinterTool.ahk

Drag Tools

There are no menus or GUI forms associated with the DragTools code, so there are no screenshots either. There are three components of this script: 'Window Mover,' 'Window Resizer,' and 'Up Down Left Right RightClick-and-Drag Actions Tool.' These are all disabled by default. See below to turn them on.

Window Mover: While holding down the *Alt* and *Ctrl* keys, simply click-and-drag a window. Windows' default behavior already allows you to move a window by dragging its title bar. The Window Mover tool simply extends this functionality so you can click anywhere to drag the active window.

Window Resizer: This also uses the *Alt* and *Ctrl* keys, but happens when you *right-click* and drag the window. When this occurs, the X and Y locations (top/left) of the window stay in the same location, but the W and H values change, such that the bottom and the right edges of the window move with the mouse cursor.

Up Down Left Right: The code for this is activated via a *left-click-and-drag* when it occurs without modifier keys. It is noteworthy that, since there are no modifier keys (Alt, Shift, Win, Ctrl), this effectively "hijacks" your mouse left-click. An attempt is made to prevent this inadvertent hijacking. In the code are the variables:

```
IgnoreDuration := 100  ; Ignore if right mouse button down shorter than this many milliseconds.
IgnoreLength := 100  ; Ignore drags less than this many pixels long.
```

The idea is that, if you intend to do a right-click gesture, you'll inevitably hold the right button down longer than 100 milliseconds and you'll drag more than 100 pixels. If neither of these is the case, then the script bypasses the assigned commands and simply sends a "right-click" command to your computer. This tool was inspired by the excellent and powerful *MouseGestureL.ahk* tool. MouseGestureL has a steep learning curve and has many features that I don't use. I found that all I really used it for was: drag-up for Home key, drag-down for End key, drag-left for Ctrl+Home and drag-right for Ctrl+End. MouseGestureL is overkill for such a limited usage, so I made this script, which recognizes only these four gestures. If a person wanted to, they could go to the code, which is at *AutoCorrect2\Tools\Integrated\DragTools.ahk* and assign different commands. It is also noteworthy that the script attempts to detect if the active window is a *PDF XChange* window and, if so, will send alternate keypresses.

<u>Known issues:</u>

-The up/down/left/right code will occasionally throw an error, when trying to detect the active window. I am still working on this.

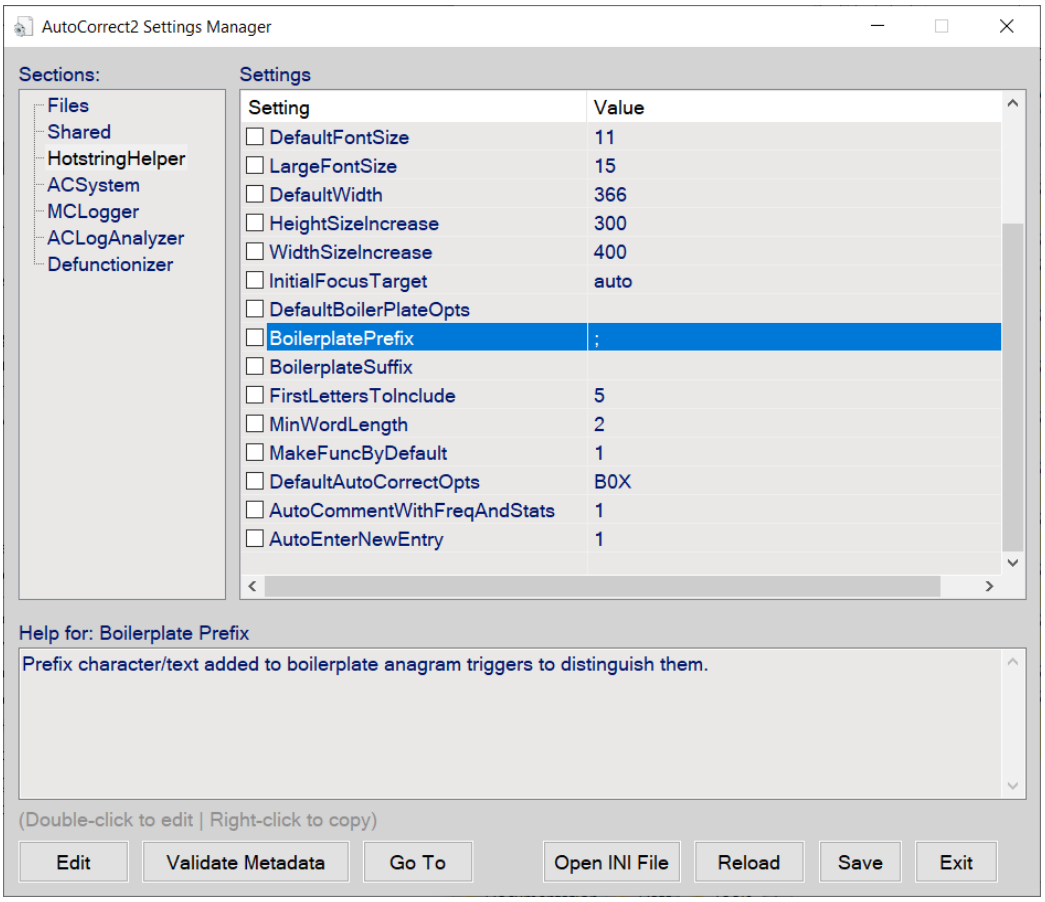
-The up/down/left/right code has caused problems for at least two users, whereby the default behavior of mouse right-clicks gets messed up. To remedy this, recent version of AutoCorrect2.ahk includes toggles to turn off the move/resize and/or the dragTools modules. In the [HotstringHelper] section of the acSettings.ini file are the keys, **EnableDragTools** and **EnableMoveResizeTools**. Set the keys to 1 to turn the tools on. This can be done via the ini file or the Settings Manager tool.

Personal Hotstrings

I have several hotstrings that I don't share. This is simply because they are so obscure and niche to my personal use that I don't think other folks will have a use for them. Also, there are several boilerplate template items that are specific to my job, that other users won't have a use for. I used to cull them from the code every time I shared it, but eventually I just put them in a separate .ahk file and don't share the file. That file is *AutoCorrect2\Core\PersonalHotstrings.ahk*. Use that location for your own unique and personal hotstrings and scriptlets, if you wish. It should be noted that—by default—the Validation functionality of hh2 does not scan this area when looking for duplicate or conflicting hotstrings. It only scans the HotstringLib.ahk file. To include PersonalHotstrings.ahk in validity scans, enable dual library support (see AC / BP section, above). Neither the MCLogger.ahk, nor other scripts that reference your hotstring library read from the PersonalHotstrings.ahk file. Also note that, in the ac2 code, the HotstringLib.ahk file is #Included as a location before the PersonalHotstrings.ahk file. This means that if you did have strings with duplicate or conflicting trigger strings, it is likely that only the first one (in HotstringLib) would be registered. The second occurrence would be ignored.

The Settings Manager Tool

Toward the end of 2025, it became apparent that having so many user-definable configuration settings embedded in the code of the various applications was problematic. Some of the main files in the AutoCorrect2 suite get updated fairly often. If a user has customized the settings, then this has to be redone with each update. Off-loading the user settings into a Configuration (.ini) file remedies this so some degree. The downside of this, is that the script files are less "self-contained," and have dependencies instead. But that's okay. Long ago, My AutoCorrect script—library and all—was all embedded in one .ahk file. That time is long-passed though, and there are now many inter-related files working together in the package. Many Windows applications use Settings.ini files. The ones that don't typically use the Windows Registry for the same purpose. None of the AutoCorrect2 apps touch the Registry at all. In order to differentiate the settings file, it is called acSettings.ini, and is located in the Data\ folder. The ini file has some comments in it to help explain things, and can be (carefully) edited directly. The recommended way to edit it, is via the SettingsManager.exe tool. In the Control Panel of hh2, look for the button with the gear icon.

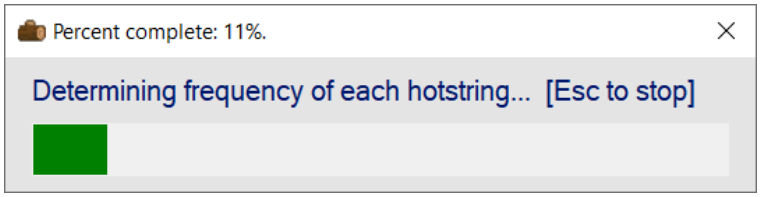


The SettingsManager.ahk code was designed to be flexible enough for other users to integrate is into their own AutoHotkey projects—those

which utilize a Settings.ini file. As seen in the picture, the [Section] categories are populated into the Tree Pane on the left of the dialog. Selecting a section will load its ini key names and values into the ListView Pane on the right. The short, wide pane on the bottom is the Help Pane. It displays information about the selected key. Double-clicking a key in the ListView will open it in a smaller dialog for editing. Depending on the key data type, the Edit Dialog will be a Windows Colorpicker, Boolean radio buttons, a File-picker, a ListBox of items, a SpinnerBox number picker, a Hotkey dialog, or a Text box. The different types can have their own validation rules and other properties. These properties, the data types, the help message, and other things are encoded in the required accompanying acSettingsMetadata.json file.

Manual Correction Logger (MCLogger)

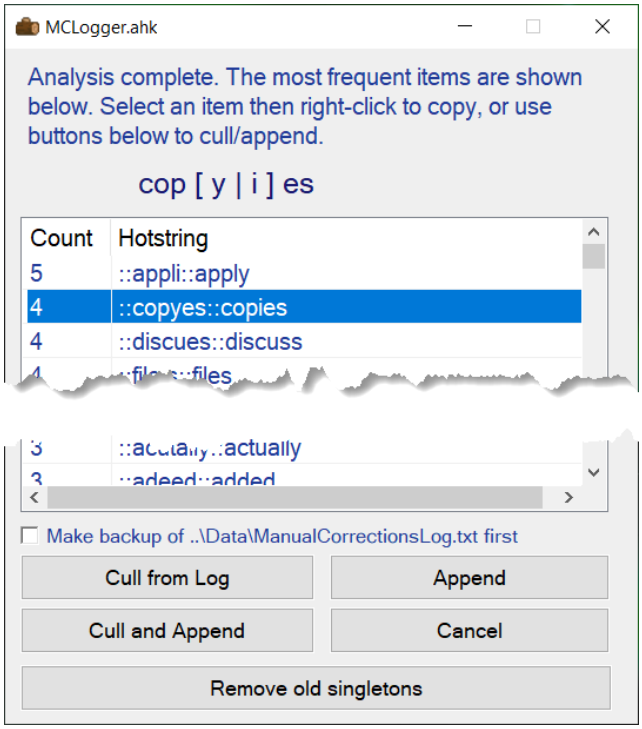
The description "***manual correction logger***" is meant to convey that it ***automatically*** logs your ***manual corrections*** as you type. The script was made to record my own reoccurring manual corrections, so if any particular reoccurring typos keep happening, I can potentially make them into new AutoCorrect library items.



The script is mostly independent of *AutoCorrect for v2* or *HotstringHelper,* and can be used even if neither ac2 or hh2 are used, though if no AutoCorrect Hotstring Library file is found, the append function will simply create one. There is a preference option, in the code, to either append the selected hotstring or to, instead, open it directly in hh2. Of course, AutoCorrect2 does have to be installed for this feature to work. AutoCorrect2 was set up to allow this feature on 5-4-2024, so please use a version at least that new. The preference option is in the acSettings.ini, where [MCLogger] has its own section. This can also be accessed via the Settings Manager tool. The key is called **SendToHH**. Set it to 0 (zero) to disable the feature. Please note that the hotstring is sent to HH via a command-line parameter that is detected by AutoCorrect2.ahk. MCLogger has no way to confirm that the parameter has been received, so if you have "SendToHH=1" but the receiving AutoCorrect file doesn't have this feature, the hotstring will be lost. It will not be appended.

Note also, that MCLogger also uses the same WordList that is used by the Exam Pane in hh2. There is also a simple validity check, against your library of autocorrect strings, "**HotstringLib.ahk**," or whatever you call yours. And for previously-removed hotstrings "**RemovedHotstrings.txt.**" So the files needed for this tool are:

Core\MCLogger.ahk
Core\MCLogger.exe
Core\HotstringLib.ahk
Core\AutoCorrect2.ahk (if you want to use 'SendToHH')
Core\AutoCorrect2.exe (if you want to use 'SendToHH')
Data\ManualCorrectionsLog.txt
Data\RemovedHotstrings.txt
Data\GitHubComboList249k.txt
Includes\AcMsgBox.ahk
Core\ColorThemeSettings.ini



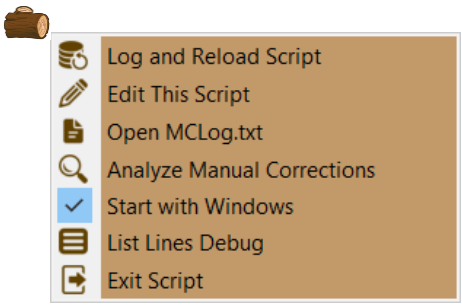
In addition to the progress bar seen in the above image, is the Report Dialog seen to the right. The report is sorted primarily by frequency of occurrence (Count) and then alphabetically.

There is a button to "Cull from Log" which removes all occurrences of the selected item from the log file. Because this process will (as suggested) remove the item from your log forever, an option is added to make a backup of the log file first. It is noteworthy that, as indicated above, the item can optionally be sent directly to Hotstring Helper via the 'Append' or the 'Cull and Append' buttons; but this works by restarting the AutoCorrect2 script and sending a command line argument. Another (more efficient) way to send the hotstring to hh2 is to select an item from the ListView, then right-click it to

save it to the clipboard. After doing so (while the MCLogger window is still active), press the hh2 hotkey (Win+H). This will also open the item in hh2, but it skips that part where the AutoCorrect2 script gets restarted, which makes it a tiny bit faster.

As discussed below, several systems are in place to avoid logging un-useful strings. Despite that, I still log many manual correction items. Of course this will be different for other users, depending on their typing patterns. By the time my manual corrections log (MCLog.txt) was only three months old, it already had 2,200+ logged potential hotstrings. Presently, there are 5,340 items. And…. Many have already been removed, because the highest-frequency reoccurring items have all been converted to new autocorrect entries in my *HotstringLib.ahk* file, and culled from the log on many occasions. Because of the high number of items, running a frequency report used to take a long time before the code was refactored by Claude AI. And… Since I keep removing the high-frequency items, what is left are a bunch of old single-occurrence "*singleton*" strings. Of course, any of the strings are singletons *at some point*—So it doesn't make sense to remove *every* singleton. But it does make sense to bulk remove *really obscure* ones that will never have more than one occurrence. But how can we bulk-remove really obscure ones? Answer: I don't know, so I made a function to remove all singleton items that are older than X number of days. How many days-old is too old? Define it in the ini file with the key **AgeOfOldSingles.**

Another Gui element is the context menu that appears when right-clicking the Windows System Tray icon.



It is recommended to put a link to the script in the Windows Startup folder* and let it run in the background. There is an option for this in the SysTray Menu as well.
*Though as indicated elsewhere, if you wholesale replace your entire AutoCorrrect2-main folder with new versions and don't save your log files, there's not much point collecting log data.

How it works

The MCLogger will watch for typing patterns that indicate you've typed a word and manually corrected the word. With each relevant keypress, it will analyze the cached characters and attempt to determine if a trigger-replacement pair can be garnered from the characters. For the purpose of identifying possible autocorrect items, we don't care about digits or other characters, so those are ignored. Similarly, if the user uses the arrow keys to move back in a word, or clicks inside of the word, then the chain of keyboard presses will have been broken. As such, these actions will reset the "typoCache". The Cache looks for letters, the BackSpace key, and the Space Key. To catch typos at the end of a sentence, periods are also grouped with letters. The watched-for pattern is: *At least three letters, followed by at least one backspace, followed by at least one letter, followed by a space*. This typoCache resets to blank if/when: a match is detected, the user clicks the left mouse button, or they press the Esc, Left, Right, Up, or Down key.

Early versions of the tool collected a lot of useless information and a lot of duplicates, so I added a filtering process. Trigger-replacement pairs are only kept if all of the following five conditions are met: (1) The replacement string is a match to a word from the word list file that is assigned near the top of the code, (2) the trigger string is NOT a word from the list, and (3) the current captured trigger-replacement pair is not the same as the previous one—to avoid duplicates. Condition (4) was added after I realized that, existing autocorrect items can still be logged. Consider this example: If I have an autocorrect entry

```
:*:creasion::creation
```

If I type the entire trigger string, then the word will be autocorrected, and I won't manually correct it. But it is possible that I'll type part of the trigger, then realize my error, such that my keypresses are this

```
creasio{Backspace}{Backspace}{Backspace}tion{Space}
```

This would not trigger the autocorrection item, but it would match the pattern the MCLogger watches for. I might then, inadvertently add the new hotstring "::creasion::creation" to my library, resulting in duplicate or conflicting items. To minimize the likelihood of this, I added condition four: If the trigger for the proposed to-be-logged hotstring conflicts with an existing autocorrect entry, don't log it. Condition (5) was added more recently. To prevent "bad" hotstrings from getting added to my library, then removed for being bad strings, then inadvertently added again, a list of removed hotstrings is now kept. Condition five is that the to-be-logged trigger item must not exist in the removed items list.

```
Current typoCache:
s<asdf<asdfg<<sdf
============================
Current Saved up Text:
2024-04-16 -- ::frin::find
2024-04-16 -- ::digigs::digits
2024-04-16 -- ::backspcae::backspace
2024-04-16 -- ::thius::this
2024-04-16 -- ::reiht::right
2024-04-16 -- ::pariicular::particular
2024-04-16 -- ::collocted::collected
2024-04-16 -- ::condictios::conditions
2024-04-16 -- ::chahes::caches
2024-04-16 -- ::stcipt::stript
2024-04-16 -- ::strtarted::restarted
```

Trigger-replacement pairs are saved up in memory, then every X minutes (10 by default), they are logged to the ManualCorrectionsLog.txt file. The "save intervals" stop if nothing is in the "saved up text," and start again when there is. There is an *OnExit* command that will also append any saved-up text if/when the script is exited or restarted in the middle of an interval. This interval system is meant to reduce the number of disk-writes, though with modern solid-state drives, there is probably no benefit to the intervals.

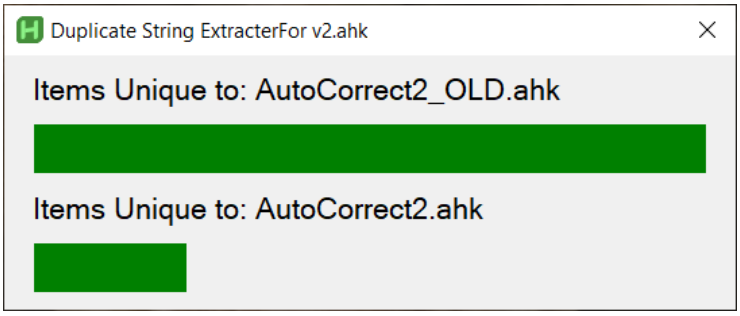
Pressing **Shift+Win+Q** will provide a popup tooltip with the current cache and saved-up text, as seen in the screenshot to the right. Pressing **Shift+Ctrl+Win+Q** (or selecting the menu item, or selecting the button in hh2's secret control panel) will analyze the data, checking for the frequency of occurrence for the items and sorting by frequency. The report will (optionally) be saved to the Windows Clipboard, and will appear in the form seen above with the radio button list. Like with DateTool-H, the nice big tooltips are made possible by Just Me's ToolTipOptions Class. With the two scripts (DateTool-H and MCL), the ToolTipOptions Class code is embedded in the script, so there is no need for an **#include** directive in the code. The hotkeys, as well as several options, can be changed via the variables near the top of the MCLogger.ahk code.

Unique Strings Extractor Tool

If you like to add your own custom AutoCorrect hotstrings, when you are manually updating your script, this tool might be useful.

From a recent forum post:

It occurred to me that if people are using older versions of ac2, they might have added lots of cool new custom autocorrect entries via the Hotstring Helper tool. If those new custom hotstrings are still at the bottom of your ac2 file, then no problem... Copy-n-paste the new items over to the newer versions of ac2 as they are released... However... Some folks might like to alpha-sort their hotstrings. (At least that's what I do--LOL) If that is the case, how can you "bring over" your new custom items to the latest release of the ac2 script?



That's what this first script is for...

IMPORTANT:
- I left the file path to my own ac2 file as an example, but delete that, and use your own scripts and related directory/paths.
- Remember to scan the .ahk files, not the same-named .exe files.
- If you kept the name "AutoCorrect2.ahk", then you might rename that to something like, "AutoCorrect2_OLD.ahk"
- Put the newer version and the older version in the same folder as this script, so that the folder contains the files:
Duplicate String Extractor forV2.ahk
AutoCorrect2.ahk
AutoCorrect2_OLD.ahk
- If they are all in the same folder, you can probably just have the file names in the code, and not the whole directory path.
- If the Unique String Finder works correctly, It will create a text file with a list of the unique items, then it will open that text file and

beep twice.
- Please note that this will not identify any unique Hotkeys or code that you added... Only hotstrings.
-Important: Hotstrings that are multi-line Continuation Section style hotstrings won't get extracted correctly either.

Conflicting or Nullified Hotstrings Finder Tool

This script is also known as the *Dead String Locator*. While the above, (*DuplicateStringExtractor2.ahk)* compares two scripts, identifying hotstrings that are unique to only the one assigned to the **extraFile** variable, *this one (ConflictingStringLocator.ahk)*, however, does an intra-analysis of a single script. It takes a lot longer to run though. The code used is the same as that of the Validation Checker of hh2, discussed above. Each hotstring is successively compared against each item in the list, so 5k autocorrect items requires about 25 million passes. With each pass, five different conflict scenarios are checked for. My current autocorrect script has about 5400 items, and it takes just under 5 minutes.

Note that the script can't identify "triplicates." If there are three of the same item, it will flag them as two duplicate pairs. Also, with each pass (loop, actually), if a duplicate or conflict is found, the script immediately goes to the next item. This makes the script faster, but it means that, if a hotstring conflicts with other items in multiple different ways, the script might only identify the first problem. Running the script a second time might be necessary to find any additional conflicts.

Depending on your goal, you'll probably want to NOT scan the Nullification Hotstring items at the top, the Do Not Sort section under that, or the Accented Words at the bottom. As such, you can set the two variables near the top of the code.

ACitemsStartAt := 1980 ; <--- "AutoCorrect Items Start At this line number."
ACitemsEndAt := 6955 ; <--- stop comparing after this line number. No point scanning the non-English accented words.

Also, be sure to ensure that the **targetFile** variable is assigned the correct path.

When the script has completed, it will beep twice, write the report to a text file (saved in the same folder as the script) and then open that text file.

Defunctionizer Tool

This tool was recently added because I know there are some users who want the HotstringLib.ahk library and the HotstringHelper tool, but really don't have a use for the f() function features. From the code comments:

```
; ==============================================================================
; DEFUNCTIONIZER
; kunkel321
; version 3-2-2025
; It removes the function call from AutoCorrect2 hotstrings.
; For example             :B0X*?:useing::f("using") ; Fixes 78 words
; is converted to:  :*?:useing::using ; Fixes 78 words

; ==============================================================================
; QUESTION:  Why remove the function calls?
; ANSWER:  Because you don't use any of the functionality.
; The f() function does three things:
; (1)Rarify/Optimize hotstring execution by only backspacing the minimum number
; of characters. But... Newer versions of AHK v2 do this anyway...
; (2) Utilizes Descolada's InputBuffer class.  This alone, is a good reason to
; keep using the f() functions!
; (3) Logging autocorrections for later analysis.  This is the original reason
; for even creating the f() function.  Not everyone wants to log/analyze though.
; If you don't care about any of these functionalities, there is no point using
; the function calls, so defunctionize the autocorrect items and use AHK's built-in
; auto-replace hotstring functionality.
```

There is no GUI form associated with the Defunctionizer app. Just set the variable at the top: **myFile := "..\..\Core\HotstringLib.ahk"** then run the code. A new defunctionized "LibDeFunct-" version of the file will be created in the same folder as the original library. Get rid of (delete) the original version, and rename your new non-function version to "HotstringLib.ahk".

It is relevant that, if a person doesn't use the f() functions, then it would make sense to change some additional options in other files. In the *AutoCorrect2.ahk* code, near the top is

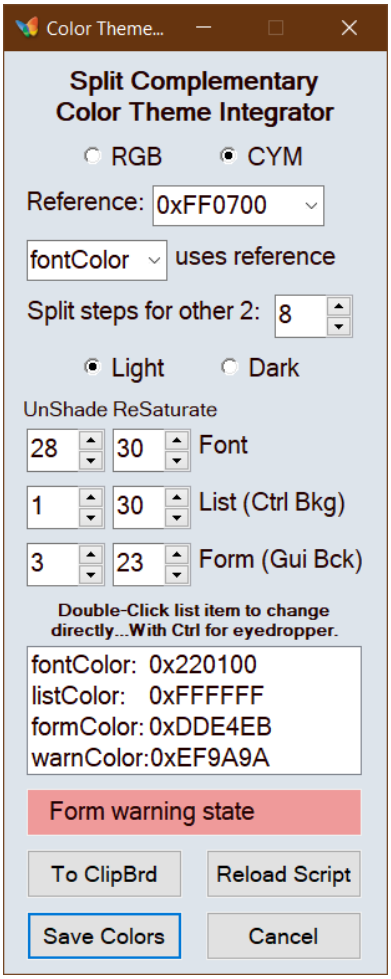
```
; ===== AutoCorrect Options =====
static DefaultAutoCorrectOpts := "B0X" ; Default options for autocorrect entries
static MakeFuncByDefault := 1        ; Check "Make Function" box by default?
```

Remove the B0 and X and change the 1 to 0 . Also, near the top of *AutoCorrectSystem.ahk* is

```
Global EnableLogging := 1   ; Main switch.
```

Change that to 0 to turn off the loggers.

Color Theme Integrator Tool



From the code comments in the **ColorThemeInt** script: The WhiteColorBlackGradient function is based on ColorGradient() by *Lateralus138* and *Teadrinker*.
Calling the Windows color picker is also based on Teadrinker code.

I Used *Claude.ai* for debugging several parts and doing the "split complemplementary" math. Some terminology is from https://colordesigner.io/color-wheel

It is a Color Theme "***Integrator***" because is writes the colors to an ini file, which several of my other AHK gui-based tools (including Hotstring Helper2) read from.

The top "Reference" combobox reads from the colorArray, which has 120 elements, equidistantly circling the color wheel. I've attempted to create two color wheel options:
* RGB uses "additive/light-based" color gradients.
* CYM is uses "subtractive/pigment-based" color gradients.
Switch between these with the top radio buttons. Both arrays start at red, then loop around, back to red.

Using terminology from the above colordesigner site...

Given a reference color, its *Complementary* color will be directly across from it, on the opposite side of the color wheel. If, instead of choosing the Complementary color, you choose the two colors that are on equal-and-opposite sides of the Complementary color, then the three points will comprise a *Split Complementary* color set. The below tool uses the color selected in the combobox as the reference color and determines which color in the colorArray is its Complementary color. The first up/down box in the gui (*Split Size*) defines the number of steps from the Complementary position, that the other two colors should be.

The main three theme color variables are used:
* **fontColor** is the color of the text on the gui and in applicable controls. I.e *myGui.SetFont("c" fontColor)*
* **listColor** is the background color of the listView, and other applicable controls. I.e. *BackgroundColor* as appears in the gui control options.
* **formColor** is the GUI background color. I.e. *myGui.BackColor := formColor*

The fourth color is "outside of" the theme:
* **warnColor** is the color used in place of the formColor, when the gui/app is in an alternate state.

The colors are assigned by default as:
* *fontColor* = The reference color that appears in the comboBox.
* *formColor* = The color that appears in the posing Counter-Clockwise from the Complementary position.
* *listColor* = The color that appears in the posing Clockwise from the Complementary position.

This means that the color which is chosen in the top comboBox corresponds to the font color of the form. The gui element (font/list/form) which is used for the reference color (and which corresponds to the top comboBox) can be changed with the second comboBox "... Is Reference."

The fourth color, *warnColor*, is not part of this color pattern scheme and is only changed via double-clicking the warnColor item on the bottom of the ListBox. However, when changing the light/darkness of the *formColor*, the *warnColor's* level of light/darkness will change with it. IIf you don't have any projects with guis that use alternate Gui Backcolors, then you can ignore the warnColor.

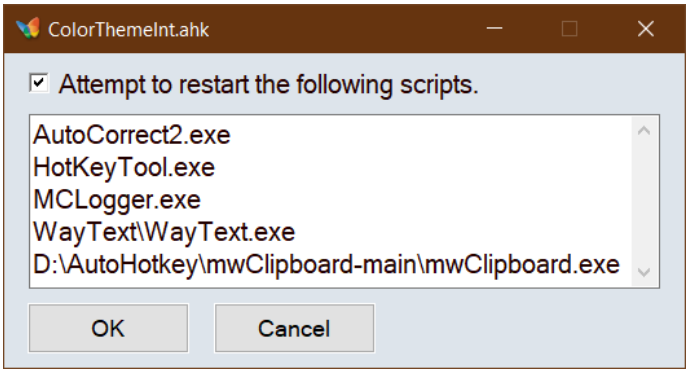
So… If the *Split Size* is 10 to 15 or so, then the pattern will be *Split Complementary.* If the Split Size is set to 0, then the listColor and the fontColor will be at the same position and will be *Complementary* to the formColor. If the Split Size is the max of 60, then all three colors will be at the same location and the theme will be *Monochromatic*. If the Split Size value is around 45 to 50, then the pattern will be "Analogous" (again, see colordesigner site). And if the Split Size is 20, then the pattern will be a "Triad." As indicated above, the primary color is set in the top comboBox, and the other two are the split colors. The split colors can swapped by setting a negative number as the Split Size.

It is noteworthy that, with a gui, the font needs to "stand out" from the background color of the gui and the controls, so the colors chosen will often need to be adjusted in terms of *lightness/darkness*. Typically the font will be on one end of the light/dark continuum, and the background colors will be at the other end. The Light/Dark radios swap this. The bottom three up/down boxes are for fine-tuning the light/darkness. The warnColor must be manually changed, but it will change light/darkness as the formColor does.

Similar to changing the shading, users may wish to "tone-down" one or more of the colors by reducing the *saturation*. This is simulated by fading the color to gray. The saturation up/down controls are next to the shading ones. Double-clicking the **ReSaturate text label** will reset the saturation levels to max. And double-clicking the **UnShade text label** will set the shade spinners to the middle number (neither light, nor dark.)

Near the bottom is the 4-row ListBox. This shows the hexadecimal color values that will get exported. The ListBox is also an "override." If you can't get a color you like with the *Hue*, *Shade*, and *Saturation* controls, **double-click** the row in the list to pick a totally different color. This will call a Windows color picker (*based strongly on Teadrinker code*) and disregard the above gui controls. Holding the **Ctrl Key** while double-clicking will allow you to "*eyedropper*-pick" a color from the screen. Note that if you then change the Hue, Shade, or Saturation controls, those controls will, again, take presidence and determine the colors used for the first three items (font, list, form), thus you will lose your custom-picked color. A warning will appear before this happens.

The current four colors, as well as the values of all the controls, can be saved to a configuration (*ColorThemeSettings.ini*) file. The tool will attempt to read from the file at start. If the ini file is not found when the script starts, default values will be applied. Near the top of the code is the "**restartTheseScripts**" list. The tool will look for a list of scripts to restart (so that their gui colors will be updated.) When the save button is pressed, the user will be given the option to also restart these scripts. Uncheck the checkbox to bypass restarting the other scripts. (See also screenshot on right.) Please note that the ColorThemeInt tool won't intelligently find the scripts that need to be restarted—that list must be manually added to the *restartTheseScripts* variable in the code.



Hotstring Helper2 will attempt to check if the *ColorThemeSettings.ini* file exists, and if it does, will add an additional button to the bottom of the *Secret Control Panel*. The button starts the *ColorThemeInt* application and launches the main form directly. This allows users to change the color of the hh2 form. The script looks for command line arguments when it starts. If there are command line arguments, the gui is shown at statup, and Esc or closing the form exits the app. this was added so that the tool could be launched from the Hotstring Helper2 tool.

Getting Set Up and Updating

It does occur to me that this section of the manual should probably be at the beginning. The Updater Tool is a recent invention though, so it got added at the end. As indicated in the Readme that is seen at the bottom of the main GitHub Repository page Github.com/kunkel321/autocorrect2 I don't really make traditional compiled "Releases." As such, I recommend expanding the blue button near the top that shows **[<> Code]** with a drop-arrow. At the bottom of the drop-list is "Download Zip." Download that and put it somewhere convenient on your computer. A shortcut to the download is here:

https://github.com/kunkel321/AutoCorrect2/archive/refs/heads/main.zip

After downloading it, right-click the zip file and choose "Extract" to unzip the file. Hopefully you'll get a folder with contents similar to the below Folder Tree. Go to *Core*\ then double-click *AutoCorrect2.exe* and you'll hear a startup announcement, which is a computery *beep beep*. You can later more the entire folder to another location if desired, but first right-click on the Windows System Tray icon (The blue Greek Psi) and choose "Exit Script." Do not change the folder structre of the subfolders or the files.

Once you have a working setup, for subsequent updates, you have a few options.

Wholesale Update

Option one is to repeat the above steps to get a clean default installation. This has the advantage of being easy, but it will cause you to lose any customizations, including any new custom hotstrings you've added.

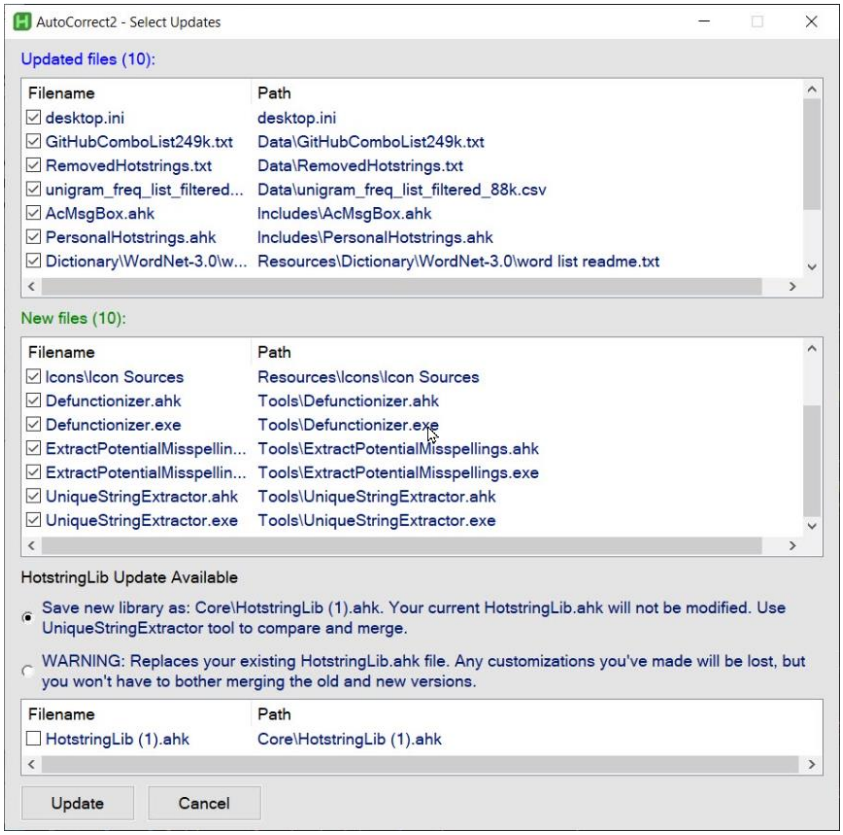
Download Zip, but keep Customizations

Another option is to drag your HotstringLib.ahk file, your acSettings.ini file, and any other files you've customized, to another location. Then download the new AutoCorrect2-main.zip and unzip it. Then delete those files and drag your customized ones into the correct location. This will allow you to keep any hotstrings you've added. Most of the code in the AutoCorrect2.ahk file is used for HotstringHelper, and HotstringHelper is primarily for adding new hotstrings to your hotstring library. So hopefully most users are doing this. The various ahk files used to have their respective configuration options set as variables near the top of the code. Most of those have been off-loaded to the acSettings.ini file to allow users to update the ahk files without losing their custom config settings.

Keep AutoCorrect2 Folder, but download new AHK files

Yet another option would be similar to the previous one, but the user keeps the folders and files and just "swaps out" the updated ahk files as they become available. This would be difficult though, since the files get updated pretty often, there are a lot to keep track of, and they must always be put in the correct location.

Use the Updater Tool



Hopefully at least some people are utilizing the **Updater Tool** at *Tools\Updater.exe*. There is also a button for it in the Control Pane. The tool was made to check GitHub for updates and compare the files with the ones on your computer, then update the appropriate files. The dialog is shown in the screenshot on the right. The tool creates a file, Date\LastUpdateCheck.ini. It checkes the "API endppoint" URL at GitHub to see the latest commits to the AutoCorrect2 repo, then compares the commit date to the last update check. If the repo has been updated, it downloads the AutoCorrect2-main.zip file, and unzips it behind the scenes, then checks the datestamp and size if each file, comparing it to those on your computer. The system is not perfect—but it seems to work fairly well. As seen in the screenshot, there is a checkbox list for **Update Files, New Files**, and the **HotstringLib**.

Updated Files: This shows files that are newer than those on your computer. In the screenshot, the files are all checked, but those files which are in the Data\ folder will be unchecked by default. These are files that are likely to have customizations that you don't want to lose. Sometimes it is necessary to update the acSettings.ini file. This is the case if/when new keys have been added. An interesting snafu is that the Updater.exe file can never be updated by this tool, because the file will always be "in use" and therefore will be locked by Windows. Updater.ahk can be updated though. When files are unchecked, and the "Update" button is pressed, the "Downloaded-but-not-installed" files will be saved and the next time the tool is used that user will be given the option to either install them for clear the memory.

New Files: This shows files that are present in the GitHub repo, but don't already exist in your AutoCorrect2-main folder. It is noteworthy that the Updater tool cannot detect when a file has been moved to a different folder. For example the PersonalHostrings.ahk was recently moved from the *Includes*\ folder to the *Core*\ folder. The Updated tool merely recognized it as a new file in the Core\ folder. It didn't remove the copy that was in the Includes\ folder.

HotstringLib: The HotstringLib.ahk file is a unique case. The assumption is that other users might add their own custom AutoCorrect hotstrings, and possibly remove ones that are problematic. I will have done the same thing with mine, and uploaded it to GitHub. Those two version will need to be merged. A simple "add any that are missing" merge won't work though, because either version may have items that were removed. A simple merge would erroneously put them all back. Therefore, the Unique String Extractor Tool was made to allow useres to get a list of strings that are unique to each version, and manually keep the ones they want. To facilitate this, the Updater tool will—when a new version of the library is found—offer two radio buttons. The top (default) one will keep the local copy of *HotstringLib.ahk* and the new version will be downloaded to the same folder and named *HotstringLib (1).ahk*. The Unique String Extractor tool will then look for those two files by default. The bottom radio button option merely replaces the local copy of the library file with the version from GitHub. This will be desired method for folks that don't customize their own AutoCorrect libraries.

Use GitHub Desktop

Advanced users that are proficient with GitHub Desktop may wish to just use that for obtaining updates. Be mindful of not overwriting any customizations to your HotstringLib.ahk file though.

Folder Structure and Files

Below is a folder tree, showing the files in the download. Brief descriptions are added.

```
AutoCorrect2-main/  (The main folder.  This is the one that gets unzipped.)
├── 🗀 Core/  (Contains the primary files.)
│   ├── AutoCorrect2.exe  (A renamed copy of Autohotkey.exe.  Clicking this runs AutoCorrect.ahk.)
│   ├── AutoCorrect2.ahk  (The main app.  About 10 other files are #Include'd in this one.)
│   ├── HotstringLib.ahk  (The AutoCorrect library.)
│   ├── MCLogger.exe  (The Manual-Correction Logger.  It's a separate app from AutoCorrect2.)
│   ├── MCLogger.ahk  (The code that gets read from the above exe file.)
│   └── PersonalHotstrings.ahk  (An optional file to hold boilerplate hotstring definitions, etc.)
│
├── 🗀 Data/  (Mostly these are the files that will be customized and unique to different users.)
│   ├── ACLogContinuous.txt  (A static version of the AutoCorrectLog.txt file that is used for the AC Stats charts.)
│   ├── acSettings.ini  (The main settings file.  Several apps read from this.  The Settings Manager tool read/writes this file.)
│   ├── acSettingsMetadata.json  (Used by the Setting Manager tool.  It holds metadata about the settings.ini file.)
│   ├── AutoCorrectsLog.txt  (The main log that gets read by ACLogAnalyzer.)
│   ├── colorThemeSettings.ini  (Used by the ColorThemeSettings tool.  Many of my GUIs get their color theme from here.)
│   ├── ErrContextLog.txt  (Keeps lists of words typed just before/after an AC was backspaced.)
│   ├── GitHubComboList249k.txt  (The big wordlist used by the Exam pane and other things.)
│   ├── LastUpdateCheck.ini  (Created by the Updater tool, so see if you have the latest version.)
│   ├── ManualCorrectionsLog.txt  (Used by the MCLogger tool.)
│   ├── PersonalApiKey.ini  (Only appears if you are using the ChatGPT word lookup module. Hold OpenAI key.)
│   ├── RemovedHotstrings.txt  (A list of AC hotstrings you've removed.  Prevents accidentally readding them.)
│   └── unigram_freq_list_filtered_88k.csv  (Word frequency list used for the WebFreq[n] data.)
│
├── 🗀 Debug/  (Several tools have a debugger option.  It enabled, logs go here.)
│   ├── Debug Folder Readme.txt
│   └── Updater_debug.txt
│
├── 🗀 Documentation/
│   ├── AutoCorrect2Manual.pdf  (This user manual.)
│   └── hh2QuickSheet.pdf  (A Quick Reference Sheet for HotstringHelper.)
│
├── 🗀 Includes/  (Holds most of the "#Include" files. These are modules used by AutoCorrect2.ahk and other things.)
│   ├── AcMsgBox.ahk  (Support for the big easy-to-read message boxes.)
│   ├── AutoCorrectSystem.ahk  (Holds the f() function, context logger, InputBuffer class, and more.)
│   ├── ChatGptWordLookup.ahk  (Optional.  See code comments for info and use.)
│   ├── DateTool.ahk  (DateTool-H.)
│   ├── DragTools.ahk  (The up/down/left/right drag tools module.)
│   ├── HotstringQuickLookup.ahk  (For looking up log stats for an individual hotstring.)
│   ├── MoveResizeTools.ahk  (Code for the window move tool and the window resize tool.)
│   └── PrinterTool.ahk  (Code for the printer tool.)
│
├── 🗀 Resources/
```

```
│   ├── 📁 Dictionary/
│   │   └── 📁 WordNet-3.0/  (The 8 dictionary files are not shown.)
│   ├── 📁 Icons/
│   │   └── AcAnalysis.ico  (45 or so icons that are used by the menus, buttons, etc. Mostly from Icons8.com.)
│   └── 📁 Images/  (Image files used.)
│       ├── GUI quicksheet.png  (Used for the screenshot in the GitHub readme.)
│       ├── icon-16.png  (The hi-rez MsgBox icons used by AcMsgBox.ahk.)
│       ├── icon-32.png
│       ├── icon-48.png
│       └── icon-64.png
│
└── 📁 Tools/  (Holds most of the tools seen in the Control Pane of HH. Each exe has a same-named ahk.)
    ├── ACLogAnalyzer.ahk/.exe  (Analyzes the AutoCorrection log file.)
    ├── ACLogStatistics.ahk/.exe  (Created the pretty HTML chart.)
    ├── ColorThemeInt.ahk/.exe  (A standalone tool that alows creation of custom GUI color combos.)
    ├── ConflictingStringLocator.ahk/.exe  (Does an intra-script scan for conflicting of duplicate hotstrings.)
    ├── Defunctionizer.ahk/.exe  (Removes the f() from all the hotstrings in HotstringLib.ahk. Not recommneded.)
    ├── ExtractPotentialMisspellings.ahk/.exe  (Creates a summary of any hotstrings that have "but misspells" in the comment.)
    ├── HotstringLibStats.ahk/.exe  (Checks number of AC log entries for every string in HotstringLib.ahk.)
    ├── SettingsManager.ahk/.exe  (Used for editing the acSettings.ini file.)
    ├── Suggester.ahk/.exe  (Used for "untrimming" word-middle strings that were trimmed to far.)
    ├── UniqueStringExtractor.ahk/.exe  (Used to compare 2 versions of HotstringLib.ahk, for merging.)
    └── Updater.ahk/.exe  (Checks GitHub repo for new versions of the files in these folder.)
```

TL;DR

Too long? Didn't read?

Just watch the videos… Go to https://github.com/kunkel321/AutoCorrect2 and look for **Youtube Screencasts** in the Read Me section.

End

Please post a reply on the AutoCorrect for v2 forum thread, or on the GitHub/AutoCorrect2 page, if you find any errors, or have any suggestions, for the above discussed code, or for this manual.

Thanks for reading, and a big THANK YOU to the people who have helped me create this suite of tools. The blue Psicon icon, the JustLog icon, and a couple of others were hand-made by me. All the monochrome systray icons are from icons8.com.

-kunkel321 (Stephen Kunkel)